
Documentazione Tecnica

Sistema di Controllo per Drone ad Ala Fissa

*Analisi architetturale del firmware embedded
sviluppato su
piattaforma **Teensy** per la gestione autonoma di un
velivolo a pilotaggio remoto con capacità di
navigazione
GPS e controllo di volo mediante algoritmi PID.*

Piattaforma: Teensy (ARM Cortex-M)
Linguaggio: C++ / Arduino Framework
Versione FW: 2.0
Classificazione: Uso Tecnico Interno

Indice

1	Panoramica Architetture	4
2	GPS e Interfacce di Comunicazione	4
2.1	Sensoristica, Attuatori e Sistema di Comunicazione	4
2.1.1	Sistema di Posizionamento Globale (GPS)	4
2.1.2	Unità di Misura Inerziale BNO055	5
2.1.3	Barometro BMP390	5
2.1.4	Sensore Laser TF-Luna	5
2.1.5	Sensore di Flusso Ottico PMW3901	6
2.1.6	Sistema Pitot per la Misura della Velocità dell'Aria	6
2.1.7	Monitoraggio della Temperatura del Motore	6
2.1.8	Sensori di Corrente e Tensione INA219	6
2.1.9	Servocomandi Aerodinamici	7
2.1.10	Sistema Propulsivo	7
2.1.11	Ricevente Radio SBUS	7
2.1.12	Sistema di Telemetria LoRa	7
2.1.13	Sistema di Segnalazione e Sicurezza	8
2.2	Canale di Comando Uplink (Ground-to-Air)	8
3	Architettura di Inizializzazione e Validazione Sensoriale del Sistema di Volo	12
3.1	Configurazione delle Interfacce di Basso Livello e Sicurezza Passiva	12
3.2	Inizializzazione moduli	12
3.3	Calibrazione dell'Unità di Misurazione Inerziale (IMU)	13
3.4	Condizionamento del Segnale Barometrico e Zero Altimetrico	13
3.5	Azzeramento dell'Anemometro (Tubo di Pitot)	14
3.6	Telemetria Energetica (Monitoraggio di Corrente e Tensione)	14
3.7	Logica di Pre-Armamento e <i>Critical Failsafe</i>	15
3.8	Fase 3 — Gestione degli Errori Critici	15
3.9	Fase 4 — Inizializzazione dei Servomeccanismi	15
3.10	Fase 5 — Avvio del Timer PID	16
4	Routine di Validazione dell'Involuppo di Volo	16
4.0.1	Rilevamento dello stato di volo	16
4.1	Modulo di Crash Detection e Mitigazione Danni Post-Impatto (<i>gestisciSchianto</i>)	17
5	Flag di Sicurezza Disattivabili da Terra	19
6	Acquisizione Sensoriale Integrata e Navigazione Autonoma	20

6.0.1	Telemetria Attuatori (Health Monitoring)	20
6.1	Monitoraggio dell’Alimentazione e Failover del Relè	20
6.2	Cinematica e Profilo Altimetrico	21
6.3	Telemetria Laser TF-Luna e Fusione con il Barometro	21
6.4	Anemometria, Cinematica GPS e Sensor Fusion	22
6.5	Flusso Ottico PMW3901 — Stima di Velocità a Bassa Quota	23
6.6	Algoritmo di Navigazione Non-Lineare (L1 Guidance Law)	24
7	Preparazione Adattiva del Profilo Propulsivo	25
7.1	Logica di Selezione della Fase di Volo	25
7.2	Fase di Crociera	25
7.3	Fase di Avvicinamento Finale	26
7.4	Limitazione Termica del Gas (gasMaxTermico)	26
8	Lettura del Ricevente S.BUS e Selezione della Modalità	27
8.1	Acquisizione del Frame S.BUS	27
8.2	Decodifica del Canale di Selezione Modalità	27
8.3	Arbitraggio Finale: Failsafe come Override Prioritario	28
9	Macchina a Stati della Modalità di Volo	28
10	Reset dei Controllori PID al Cambio di Modalità	29
10.1	Rilevamento della Transizione di Stato	29
10.2	Procedura di Reset delle Variabili di Stato PID	29
10.3	Sblocco di Emergenza Post-Schianto	30
11	Modalità Manuale — Mappatura Diretta dei Canali RC	30
11.1	Condizione di Attivazione	30
11.2	Mappatura dei Canali RC sugli Attuatori	31
11.2.1	Gas (Canale 3 — <code>canaliRC[2]</code>)	31
11.2.2	Rollio (Canale 1 — <code>canaliRC[0]</code>)	31
11.2.3	Beccheggio (Canale 2 — <code>canaliRC[1]</code>)	31
11.3	Ottimizzazione dell’Output Seriale	32
12	Modalità Automatica e Failsafe — Controllo PID	32
12.1	Condizione di Attivazione	32
12.2	Controllo di Volo Autonomo: Architettura PID a Cascata	33
12.2.1	Gestione del Dominio Temporale (<i>Time-Step Protection</i>)	33
12.2.2	Parametri Operativi del Controllore	33
12.2.3	Loop 1: Altitudine e <i>Pitch-Targeting</i>	34
12.2.4	Loop 2 e 3: Dinamica d’Assetto (Pitch e Roll)	34
12.2.5	Loop 4: Autothrottle e Gestione dell’Energia Cinetica	35

13 Attuazione Fisica, Override di Sicurezza e Watchdog Hardware	35
13.1 Logica di Attuazione e Inibizione Motore (Crash Override)	35
13.2 Gestione Dinamica dell'Interfaccia Visiva (HMI)	36
14 Allocazione dei Comandi e Matrice di <i>Mixing</i> Adattiva	37
14.1 Risparmio Energetico Attivo (<i>Load Shedding</i>)	37
14.2 Matrice Cinematica e Riconfigurazione Strutturale	37
14.3 Gestione Dinamica dell'Hardware (Attach/Detach)	38
14.4 Saturazione Meccanica e Attuazione Finale	38
15 Lettura Seriale del Sottosistema GNSS	39
15.1 Sistema di Telemetria	39
15.1.1 Acquisizione Dati	39
15.1.2 Codifica degli Allarmi	40
15.1.3 Struttura del Pacchetto	40
15.1.4 Esempio di Pacchetto Telemetrico	42
15.1.5 Formato e Robustezza	44
15.1.6 Considerazioni Sistemistiche	44

1. Panoramica Architetturale

Il firmware è concepito come un sistema di controllo di volo (*Flight Control System*, FCS) completamente embedded, destinato a un velivolo ad ala fissa con configurazione *flying wing* a quattro superfici mobili. Il codice integra in un unico ciclo di esecuzione la lettura di una rete di sensori eterogenea, l'elaborazione di algoritmi di controllo in retroazione (*closed-loop*), la navigazione autonoma verso coordinate GPS predefinite e la trasmissione e ricezione di dati telemetrici in tempo reale tramite modem LoRa.

Il microcontrollore adottato — un **Teensy** basato su nucleo ARM Cortex-M — garantisce la potenza computazionale necessaria per la gestione concorrente di più bus seriali e I²C, indispensabile data la molteplicità dei sensori installati a bordo.

Nota Tecnica

L'intera architettura segue il paradigma *sense-compute-actuate*: i sensori acquisiscono lo stato fisico del velivolo, il firmware elabora i dati attraverso i controllori PID, e gli attuatori (servomotori e motore brushless) imprimono le correzioni necessarie alle superfici di controllo.

2. GPS e Interfacce di Comunicazione

2.1. Sensoristica, Attuatori e Sistema di Comunicazione

L'autopilota sviluppato integra una rete eterogenea di sensori, attuatori e moduli di comunicazione finalizzata alla determinazione dello stato del velivolo, alla navigazione autonoma e all'esecuzione delle leggi di controllo. I vari dispositivi sono interconnessi mediante protocolli di comunicazione digitali quali **I²C**, **SPI** e **UART**, oltre ad alcuni ingressi analogici utilizzati per l'acquisizione di grandezze fisiche continue.

2.1.1. Sistema di Posizionamento Globale (GPS)

La determinazione della posizione geografica del velivolo è affidata a un ricevitore GPS collegato alla porta seriale hardware **Serial1**. Il modulo riceve simultaneamente i segnali provenienti da più satelliti appartenenti alla costellazione GNSS e, mediante tecniche di trilaterazione, determina la propria posizione nello spazio.

Il dispositivo trasmette periodicamente messaggi conformi allo standard *NMEA 0183*, costituiti da stringhe ASCII contenenti informazioni quali:

- latitudine e longitudine;
- altitudine sul livello del mare;

- velocità rispetto al suolo;
- direzione di moto;
- numero di satelliti agganciati;
- qualità del fix.

2.1.2. Unità di Misura Inerziale BNO055

L'orientamento del drone viene stimato tramite un sensore **Bosch BNO055**, collegato al bus I²C all'indirizzo **0x28**. Tale dispositivo integra all'interno dello stesso package tre differenti sensori MEMS:

- accelerometro triassiale;
- giroscopio triassiale;
- magnetometro triassiale.

Una caratteristica distintiva del BNO055 consiste nella presenza di un microcontrollore ARM interno che esegue autonomamente algoritmi di *sensor fusion*. Grazie a tale elaborazione il dispositivo fornisce direttamente gli angoli di orientamento (*roll*, *pitch* e *yaw*) senza richiedere calcoli complessi da parte del microcontrollore principale. L'accelerometro misura le accelerazioni lineari lungo i tre assi cartesiani, il giroscopio rileva le velocità angolari mentre il magnetometro misura il campo magnetico terrestre, consentendo la determinazione della direzione rispetto al Nord magnetico.

2.1.3. Barometro BMP390

La misura dell'altitudine viene effettuata tramite un sensore barometrico **BMP390**, anch'esso collegato al bus I²C. Il principio di funzionamento si basa sulla variazione della pressione atmosferica con la quota. All'interno del sensore è presente una membrana micromeccanica MEMS che si deforma in funzione della pressione esterna; tale deformazione viene convertita in un segnale elettrico e successivamente digitalizzata. Oltre alla pressione atmosferica il sensore fornisce anche la temperatura ambiente, necessaria per compensare gli errori termici della misura. L'altitudine viene ricavata attraverso la relazione barometrica standard che lega pressione e quota. Poiché la pressione atmosferica varia lentamente e risulta relativamente immune ai disturbi ad alta frequenza, il BMP390 rappresenta il riferimento principale per la stima della quota alle medie ed elevate altitudini. Per aumentare la precisione della misura vengono utilizzate tecniche di oversampling e filtraggio digitale integrate direttamente nel dispositivo.

2.1.4. Sensore Laser TF-Luna

La misura della distanza dal terreno alle basse quote è affidata a un sensore **TF-Luna LiDAR**, collegato tramite interfaccia UART dedicata. Il principio di funzionamento ap-

partiene alla categoria *Time of Flight* (ToF). Il dispositivo emette brevi impulsi di luce infrarossa e misura il tempo impiegato dalla radiazione per raggiungere l'ostacolo e tornare al ricevitore. Conoscendo la velocità della luce è possibile ricavare la distanza. Il sensore trasmette continuamente frame binari contenenti distanza, intensità del segnale e checksum di verifica. Rispetto al barometro, il LiDAR offre una precisione nettamente superiore alle basse quote e risulta particolarmente utile durante le fasi di decollo.

2.1.5. Sensore di Flusso Ottico PMW3901

Per la stima del moto rispetto al terreno viene impiegato un sensore di flusso ottico **PMW3901**, collegato mediante interfaccia SPI. Il dispositivo utilizza una microcamera ad alta velocità che acquisisce continuamente immagini della superficie sottostante. Confrontando due immagini successive mediante algoritmi di correlazione, il sensore determina lo spostamento apparente degli elementi presenti nel campo visivo.

2.1.6. Sistema Pitot per la Misura della Velocità dell'Aria

La velocità aerodinamica viene determinata mediante un tubo di Pitot collegato a un sensore di pressione differenziale con uscita analogica. Il tubo di Pitot possiede due prese di pressione:

- una esposta direttamente al flusso d'aria;
- una collegata alla pressione statica ambiente.

La differenza tra le due pressioni corrisponde alla pressione dinamica del flusso.

2.1.7. Monitoraggio della Temperatura del Motore

La temperatura del gruppo propulsivo viene monitorata mediante un sensore analogico collegato a un ingresso ADC del microcontrollore.

La tensione generata dal sensore viene convertita digitalmente dalla Teensy e trasformata in temperatura attraverso la curva caratteristica del dispositivo.

2.1.8. Sensori di Corrente e Tensione INA219

Il monitoraggio energetico è affidato a sei moduli **INA219** connessi al bus I²C. Ogni dispositivo integra:

- amplificatore di misura;
- convertitore analogico-digitale;
- circuito di misura della tensione di linea.

La corrente viene determinata misurando la caduta di tensione ai capi di una resistenza shunt di precisione.

I moduli consentono il monitoraggio indipendente di:

- batteria principale;
- alimentazione della scheda di controllo;
- singoli servocomandi.

2.1.9. Servocomandi Aerodinamici

Il controllo delle superfici aerodinamiche è affidato a quattro servomotori digitali. I servocomandi ricevono segnali PWM generati dalla Teensy; la larghezza dell'impulso determina la posizione angolare dell'albero motore interno.

Ogni servo integra:

- motore elettrico in corrente continua;
- riduttore meccanico;
- potenziometro di feedback;
- controllore elettronico interno.

2.1.10. Sistema Propulsivo

La propulsione è affidata a un motore brushless trifase controllato mediante un *Electronic Speed Controller* (ESC). L'ESC riceve dal microcontrollore un segnale PWM equivalente a quello utilizzato dai servomotori e lo converte in una sequenza di commutazione trifase ad alta frequenza. Il controllore elettronico regola:

- corrente di fase;
- velocità di rotazione;
- coppia erogata.

In questo modo è possibile controllare con precisione la spinta prodotta dall'elica.

2.1.11. Ricevente Radio SBUS

Il collegamento con il pilota remoto avviene tramite una ricevente compatibile con protocollo **SBUS**. A differenza dei tradizionali segnali PWM, SBUS utilizza una comunicazione seriale digitale che trasmette simultaneamente fino a sedici canali di comando all'interno di un singolo frame dati.

2.1.12. Sistema di Telemetria LoRa

La comunicazione con la stazione di terra è affidata a un modulo LoRa collegato tramite UART.⁴ La tecnologia LoRa (*Long Range*) utilizza tecniche di modulazione a spettro espanso che consentono comunicazioni a lunga distanza con consumi energetici estrema-

mente ridotti. La telemetria costituisce il principale canale di supervisione durante il volo autonomo e consente anche l'invio di comandi specifici.

2.1.13. Sistema di Segnalazione e Sicurezza

Completano l'architettura elettronica alcuni dispositivi ausiliari costituiti da LED di stato, buzzer piezoelettrico e relè di sicurezza. I LED forniscono un'indicazione immediata dello stato operativo dei vari sottosistemi, mentre il buzzer permette la segnalazione acustica di errori, calibrazioni e procedure di avvio. Il relè viene utilizzato come elemento di sicurezza per l'attivazione o la disattivazione di circuiti critici in caso di emergenza. Nel loro insieme tali dispositivi costituiscono il primo livello di diagnostica locale dell'intero sistema avionico.

2.2. Canale di Comando Uplink (Ground-to-Air)

Oltre al downlink telemetrico descritto in §2.3, il sistema dispone di un canale di comando bidirezionale che consente alla stazione di terra (o a un operatore via porta seriale di debug) di inviare istruzioni al velivolo in volo. Il canale è gestito dalla coppia di funzioni `comandi_da_terra()` ed `elaboraComando()`, richiamate a ogni ciclo del loop principale.

Sorgenti fisiche. I comandi possono giungere indifferentemente da due sorgenti hardware, interrogate in sequenza a ogni ciclo:

- Il modem LoRa, sulla medesima porta seriale `TELEMETRIA (Serial4)` impiegata per il downlink;
- La porta seriale USB di debug (`Serial`), per test a banco o comando via cavo.

La funzione `comandi_da_terra()` accumula i caratteri ricevuti in un buffer fino al carattere di terminazione `'\n'` (newline), momento in cui il buffer viene passato a `elaboraComando()` e successivamente svuotato. Come misura di robustezza contro buffer corrotti o comandi malformati, se il buffer supera i 64 caratteri senza newline viene resettato forzatamente.

Protocollo. Ogni comando valido deve rispettare il formato testuale:

`CMD:CAMPO:VALORE`

dove `CAMPO` identifica il parametro o l'azione da eseguire e `VALORE` il dato associato (intero o decimale, a seconda del campo). Un comando che non inizia con il prefisso `CMD:` viene scartato senza alcuna azione.

Elenco completo dei comandi gestiti:

Campo	Effetto	Vincoli / Note
SERVO_ISX / IDX SERVO_ESX / EDX	Impone la posizione angolare diretta al rispettivo servo (interno/esterno, SX/DX).	Valore vincolato tra 45° e 135°.
SERVO_[...]_ATTACH	Ricollega (attach) il rispettivo servomotore al proprio pin PWM.	—
SERVO_[...]_DETACH	Scollega (detach) il rispettivo servomotore, liberando il segnale PWM.	—
RELE_ON / OFF	Forza manualmente lo stato del relè di ridondanza alimentazione (PIN_RELE) e aggiorna il flag <code>releAttivato</code> .	—
SICUREZZA_SCHIANTO_[ON OFF]	Abilita/disabilita il rilevamento schianto (<code>schianto_sicurezza</code>).	Vedi §5 modificato.
SICUREZZA_ALIMENTAZIONE_[ON OFF]	Abilita/disabilita il monitoraggio di alimentazione (<code>alimentazione_sicurezza</code>).	Vedi §4 modificato.
SICUREZZA_SERVI_[ON OFF]	Abilita/disabilita la diagnostica di corrente dei servi (<code>servo_sicurezza</code>).	Vedi §6.0.1 modificato.
SICUREZZA_TEMP_[ON OFF]	Abilita/disabilita il limitatore termico del gas (<code>sistema_sicurezza_temp</code>).	Vedi §7 modificato.
GAS	Impone direttamente il gas in microsecondi tramite <code>motore.writeMicroseconds()</code> .	Applicato solo se <code>global_modalitaVolo == 1</code> (Manuale); tra <code>GAS_NEUTRO</code> (1000 μ s) e <code>GAS_MASSIMO</code> (2000 μ s).

Continua dalla pagina precedente

Campo	Effetto	Vincoli / Note
MOD0	Forza la modalità di volo (1 = Manuale, 2 = Auto GPS).	Ignorato se failsafe attivo; ammessi solo 1 o 2.
SET_LATITUDE	Aggiorna la coordinata del waypoint di navigazione.	Valori tra -90.0° e $+90.0^\circ$.
SET_LONGITUDE	Aggiorna la coordinata del waypoint di navigazione.	Valori tra -180.0° e $+180.0^\circ$.
SET_ALTITUDE	Aggiorna la quota target per la navigazione autonoma.	Valori tra 0.0 m e 500.0 m.
SET_KP_VEL / SET_KI_VEL / SET_KD_VEL	Aggiornano i guadagni P/I/D del loop Autothrottle (§ 12.2.5).	$K_p \in [0, 10.0]$; $K_i \in [0, 2.0]$; $K_d \in [0, 5.0]$.
SET_KP_ROLL / SET_KI_ROLL / SET_KD_ROLL	Aggiornano i guadagni P/I/D del loop di Rollio (§ 12.2.4).	Stessi limiti.
SET_KP_PITCH / SET_KI_PITCH / SET_KD_PITCH	Aggiornano i guadagni P/I/D del loop di Beccheggio (§ 12.2.4).	Stessi limiti.
SET_KP_ALT / SET_KI_ALT / SET_KD_ALT	Aggiornano i guadagni P/I/D del loop di Altitudine (§ 12.2.3).	Stessi limiti.
SET_VEL_CROCIERA	Aggiorna VELOCITA_CROCIERA_km (§ 7.2).	$[20.0, 150.0]$ km/h.
SET_VEL_AVVICINAMENTO	Aggiorna VELOCITA_AVVICINAMENTO_km (§ 7.3).	$[15.0, 150.0]$ km/h.

Continua dalla pagina precedente

Campo	Effetto	Vincoli / Note
SET_ALT_MIN	Aggiorna (§ 12.2.3).	ALTEZZA_MIN_m [2.0, ALTEZZA_MAX_m); rifiutato se ≥ ALTEZZA_MAX_m.
SET_ALT_MAX	Aggiorna (§ 12.2.3).	ALTEZZA_MAX_m (ALTEZZA_MIN_m, 500.0]; rifiutato se ≤ ALTEZZA_MIN_m.
SET_LIMITE_TEMP_MOTORE	Aggiorna la soglia T_MOTORE_THROTTLE_END (§ 7.4).	(T_MOTORE_THROTTLE_START, 120.0)
SET_RAGGIO_WAYPOINT	Aggiorna RAGGIO_ACCETTAZIONE_MINIMO_m (§ 6.6).	[5.0, 100.0] m.
REQ_DIAG	Forza l'invio immediato del prossimo pacchetto diagnostico TEL2/TEL3/TEL4 (§ 15.1).	—
RESET_PID	Azzera gli accumulatori in- tegrali/derivativi dei quattro PID e il riferimento temporale tempoPassatoPID (funzione resettaPID(), § 10.2).	—

Nota Tecnica: Tutti i comandi SET_* elencati sopra restituiscono ACK con il valore accettato, oppure NACK con motivo «fuori limite» (o varianti «_maggiore_di_max»/«_minore_di_min» per le soglie di quota) se il valore fornito è fuori dall'intervallo consentito.

Nota Tecnica: Ogni ricezione di un comando valido (terminato da newline) attiva un breve segnale acustico di conferma (segnalaOK()), fornendo un riscontro immediato — udibile anche in assenza di telemetria visiva — dell'avvenuta ricezione lato velivolo.

3. Architettura di Inizializzazione e Validazione Sensoriale del Sistema di Volo

La fase di accensione (Power-On) di un sistema aeromobile a pilotaggio remoto richiede una rigorosa e deterministica sequenza di configurazione, nota come *Power-On Self-Test* (POST) ed inizializzazione hardware. Questa procedura è fondamentale per garantire che l'unità di calcolo principale, i bus di comunicazione e l'intera suite sensoriale raggiungano uno stato di stabilità operativa prima di autorizzare l'armamento dei propulsori. La tolleranza agli errori in questa fase è pari a zero per i sensori primari, poiché l'affidabilità della navigazione spaziale dipende interamente dalla correttezza dei valori di offset e di tara acquisiti a terra.

3.1. Configurazione delle Interfacce di Basso Livello e Sicurezza Passiva

Il primo stadio del bootloader prevede l'inizializzazione del layer fisico di comunicazione. I bus seriali sincroni (I2C) vengono configurati in modalità *Fast Mode* con un clock di 400 kHz. Questa frequenza elevata è un requisito stringente per minimizzare la latenza di campionamento durante i cicli del controllore PID, garantendo che i dati inerziali vengano acquisiti con un ritardo di fase trascurabile. Contemporaneamente, vengono aperte le porte seriali asincrone (UART) dedicate alla telemetria a lungo raggio, alla ricezione dei pacchetti GNSS e all'interrogazione dei sensori optoelettronici.

Prima di avviare qualsiasi protocollo di lettura, il controllore applica un *hardware interlock*: tutte le uscite destinate agli attuatori (servomotori e relè di potenza) vengono forzate in uno stato logico basso, garantendo una condizione di sicurezza passiva per prevenire azionamenti involontari dovuti a fluttuazioni transitorie di tensione. In oltre al termine del settaggio delle uscite vengono emessi una serie di bip sonori per certificare l'inizio del setup dei moduli.

3.2. Inizializzazione moduli

Il sistema tenta l'inizializzazione di tutti i moduli sensoriali in un ciclo **while** che si ripete fino a un massimo di **MAX_TENTATIVI = 3 iterazioni**, o fino alla riuscita completa. In ogni iterazione vengono verificati:

- **Sensore di Flusso Ottico:** L'inizializzazione verifica l'integrità del bus di comunicazione e la risposta del sensore d'immagine. Questo componente, essenziale per il mantenimento della posizione sul piano orizzontale (XY) in assenza di segnale satellitare, richiede la validazione dei propri registri interni prima di iniziare l'acquisizione dei pattern di contrasto del terreno. Nel caso di una corretta inizializzazione verranno emessi più suoni dalla funzione **segnalaOK**, nel caso contrario **segnalaErrore**.
- **Telemetria Laser (LIDAR):** Essendo un dispositivo basato sul tempo di volo (*Time-of-Flight*) interfacciato via UART, l'inizializzazione prevede una pulizia preventiva dei buffer seriali (flushing) per eliminare pacchetti frammentati generati durante

il transitorio di accensione. Successivamente, l'algoritmo si mette in ascolto di uno specifico *header* di sincronizzazione in esadecimale. Una volta allineato il parsing dei frame, il sistema acquisisce e scarta un treno iniziale di letture per permettere all'ottica di stabilizzarsi termicamente e compensare l'eventuale rumore di fondo causato dalla luce ambientale. Nel caso di una corretta inizializzazione verranno emessi più suoni dalla funzione **segnalaOK**, nel caso contrario **segnalaErrore**.

3.3. Calibrazione dell'Unità di Misurazione Inerziale (IMU)

L'unità IMU rappresenta il cuore nevralgico per la stima dell'assetto. I sensori MEMS (*Micro-Electro-Mechanical Systems*) al suo interno sono soggetti a derive termiche e rumore bianco; pertanto, l'inizializzazione richiede un processo meticoloso:

1. **Stabilizzazione del Clock:** Viene forzato l'utilizzo di un oscillatore al quarzo esterno, bypassando i clock interni del sensore per garantire una precisione di integrazione temporale superiore, fondamentale per il calcolo delle velocità angolari.
2. **Convergenza dei Filtri Interni:** Il sistema attende passivamente che i filtri di Kalman o Mahony integrati nel sensore inerziale (se presenti in logica hardware) o le routine di auto-calibrazione giroscopica completino il loro ciclo, segnalando un livello di affidabilità adeguato.
3. **Acquisizione della Tara Statica:** Sfruttando l'assunzione che il velivolo sia perfettamente fermo al suolo, l'unità di calcolo campiona un volume elevato di dati spaziali (Rollio, Beccheggio e Imbardata) in una finestra temporale definita. La media statistica di queste **IMU_CAMPIONI_TARA** letture permette di calcolare il *bias* di errore statico dei giroscopi. Durante il volo, il firmware sottrae in tempo reale gli offset calcolati esclusivamente per gli assi di Rollio e Beccheggio, prevenendo il fenomeno della deriva d'integrazione (*Random Walk*) su questi due assi. L'offset di Imbardata (Yaw), pur essendo calcolato in fase di setup, viene deliberatamente ignorato nel ciclo principale, utilizzando il dato grezzo fornito dal DSP del sensore. Durante questa acquisizione il firmware si limita a segnalare l'avanzamento tramite una sequenza di punti stampati sulla porta seriale di debug, senza attivare LED o buzzer dedicati (a differenza della calibrazione di barometro e Pitot, che utilizzano **segnalaCalibrazione** rispettivamente su **PIN_LED_BLU_PID** e **PIN_LED_VERDE_GPS**). Anche qui, in caso di una corretta inizializzazione, verrà emesso un segnale acustico di conferma dalla funzione **segnalaOK**; in caso contrario da **segnalaErrore**.

3.4. Condizionamento del Segnale Barometrico e Zero Altimetrico

La stima dell'altitudine barometrica è fortemente suscettibile alle fluttuazioni di pressione dinamica generate dalla fluidodinamica dei rotori (*prop wash*) e dai gradienti termici.

- **Filtro IIR e Sovracampionamento:** Il sensore di pressione piezo-resistivo viene configurato per operare con un sovracampionamento spinto (*oversampling*). I dati

grezzi passano attraverso un filtro digitale IIR (*Infinite Impulse Response*) integrato nell'hardware. Questa configurazione agisce come un filtro passa-basso, attenuando drasticamente i picchi di pressione ad alta frequenza senza introdurre un ritardo di fase inaccettabile per il PID di quota.

- **Tara Altimetrica (QFE):** Il sistema rileva l'altitudine rispetto al livello del mare standard (basata sull'isobara a 1013.25 hPa) ed esegue una media di **20** di campionamenti per stabilire l'altitudine locale di partenza. Durante questa calibrazione lampeggerà il **PIN_LED_BLU_PID** : segnalando l'acquisizione dati. Anche qui per evitare dati corrotti il sistema scarnerà le prime **3** tarature per poi successive attivare un controllo di validità fisica (hardware boundary check): letture che indicano quote impossibili o fuori scala innescano un errore immediato segnalata all'utente con la funzione **segnaErrore**.
- **Sensor Fusion Preliminare:** Se il telemetro LIDAR è presente, validato e rileva il suolo a una distanza coerente (< 5.0 m), il dato barometrico viene incrociato con l'offset telemetrico. Tuttavia, tale fusione è rigidamente vincolata al fatto che l'altitudine assoluta sul livello del mare (ASL), misurata dal barometro rispetto a 1013.25 hPa, sia essa stessa inferiore a 5.0 metri. Se il drone viene acceso a quote ASL superiori (es. entroterra), l'incrocio con il LIDAR viene silenziosamente ignorato e si utilizza solo la tara barometrica.

3.5. Azzeramento dell'Anemometro (Tubo di Pitot)

Per i velivoli che necessitano della misurazione della velocità all'aria (*Airspeed*), il trasduttore di pressione differenziale (Tubo di Pitot) deve essere rigorosamente calibrato in assenza di flusso d'aria dinamico. L'algoritmo acquisisce un lungo array di **100** campioni analogici che rappresentano la pressione statica differenziale a velocità nulla. La media di questi campioni costituisce lo zero strumentale. Anche qui l'utente verrà aggiotnato tramite il lapeggio del **PIN_LED_VERDE_GPS**. L'equazione di Bernoulli, che lega la pressione dinamica al quadrato della velocità, richiede che questo zero sia estremamente preciso: un offset errato, anche di pochi millivolt sul convertitore analogico-digitale (ADC), introdurrebbe errori asintotici significativi nel calcolo della velocità di stallo. Il sistema verifica che il valore di zero calcolato rientri in una tolleranza nominale del sensore, isolando potenziali guasti come occlusioni pneumatiche o derive di alimentazione. Nel caso di una corretta inizializzazione verranno emessi più suoni dalla funzione **segnalaOK** , nel caso contrario **segnalaErrore**.

3.6. Telemetria Energetica (Monitoraggio di Corrente e Tensione)

L'ultima fase diagnostica prevede l'interrogazione dei monitor di potenza basati su resistori di shunt. Il sistema verifica la comunicazione I2C con i vari circuiti integrati preposti alla misurazione degli assorbimenti differenziali. Questi sensori monitorano separatamente la

tensione e la corrente della batteria principale (destinata ai rotori/motori), l'alimentazione del microcontrollore stesso e i canali isolati dei servomeccanismi di controllo (interni ed esterni). La validazione di questi nodi assicura che il sistema possa monitorare in tempo reale il consumo energetico in Joule, prevenire le cadute di tensione impreviste (*Brownout*) e gestire gli allarmi di batteria scarica durante la missione.

3.7. Logica di Pre-Armamento e *Critical Failsafe*

Al termine del ciclo iterativo di polling sensoriale, l'architettura logica valuta una matrice di stato booleana. Il sistema adotta una politica di sicurezza di tipo *Hard-Fail* per la sensoristica primaria: l'assenza di convergenza nella calibrazione dell'**IMU**, del **Barometro** o del **Pitot** viene classificata come errore bloccante e catastrofico. In tale eventualità, il software si blocca in un loop infinito (Failsafe Lock), inibendo l'avvio degli algoritmi di controllo e segnalando visivamente e acusticamente all'operatore la necessità di una diagnostica hardware manuale.

3.8. Fase 3 — Gestione degli Errori Critici

Qualora al termine dei tre tentativi uno o più tra IMU, barometro e Pitot non risultassero operativi, il sistema entra in uno stato di **blocco permanente** (*hard fault*): il **LED rosso** rimane acceso fisso e il buzzer emette un tono pulsato a 2000 Hz (300 ms *on*, 400 ms di pausa), segnalando la necessità di intervento fisico sull'hardware prima del riavvio.

Avvertenza

In condizioni di errore critico, il firmware non cede mai il controllo al `loop()`: il velivolo rimane a terra con i servomeccanismi non inizializzati, impedendo qualsiasi tentativo di volo in stato degradato.

Tabella 2: Segnalazione LED e Buzzer per inizializzazione, calibrazione e allarmi

Situazione	LED	Comportamento	Frequenza
Successo inizializzazione sensore	Verde GPS	ON 300 ms	1
Errore sensore	Rosso Alarm	3 lampeggi	4
Calibrazione barometro	Blu PID	Lampeggio alternato	5
Calibrazione Pitot	Verde GPS	Lampeggio alternato	5
Calibrazione IMU (tara)	—	Nessuna segnalazione LED (solo stampa seriale)	—
Tentativo fallito (non critico)	Rosso Alarm	ON 2 s	—
Errore critico (blocco drone)	Rosso Alarm	ON fisso	2

3.9. Fase 4 — Inizializzazione dei Servomeccanismi

Con tutti i sensori operativi, i quattro servomeccanismi e l'ESC vengono attaccati ai rispettivi pin PWM e posizionati nelle condizioni iniziali di sicurezza:

- Superfici di controllo: posizione neutra (`CENTRO_SERVO = 90°`)
- Motore: gas idle da stick manuale (`GAS_NEUTRO = 1000  $\mu$ s`), da non confondere con `GAS_MINIMO` (§ 11.2.1)

La procedura si conclude con un *jingle* sonoro tritonale ascendente e un lampo simultaneo di tutti e tre i LED, a conferma visiva e acustica del sistema completamente operativo.

3.10. Fase 5 — Avvio del Timer PID

L'istruzione `tempoPassatoPID = millis()` registra il timestamp di inizio, inizializzando il riferimento temporale per il calcolo dei termini integrale e derivativo dei controllori PID nel loop principale.

4. Routine di Validazione dell'Involuppo di Volo

4.0.1. Rilevamento dello stato di volo

Al fine di distinguere in modo affidabile la fase di volo dalla permanenza a terra, l'autopilota implementa un algoritmo di validazione del decollo basato sulla verifica simultanea di parametri cinematici e altimetrici. L'obiettivo di tale procedura è evitare che oscillazioni temporanee dei sensori o brevi accelerazioni durante le operazioni a terra vengano erroneamente interpretate come un reale decollo. Per questo motivo il sistema non considera sufficiente il semplice superamento istantaneo di una soglia, ma richiede che determinate condizioni vengano mantenute per un intervallo temporale minimo prestabilito.

Verifica delle condizioni di decollo La procedura monitora continuamente due grandezze fondamentali:

- la velocità del velivolo;
- l'altitudine rispetto al punto di partenza.

Affinché il drone possa essere considerato effettivamente in volo devono essere soddisfatte contemporaneamente le seguenti condizioni: sia la velocità all'aria (*Airspeed*) sia la velocità rispetto al suolo (*Groundspeed*) siano contemporaneamente maggiori della soglia minima di velocità `SOGLIA_VELO_DECOLLO_MS`, e che l'altezza del velivolo sia maggiore della soglia di altezza `SOGLIA_ALT_DECOLLO_M`. La verifica simultanea di queste condizioni riduce la probabilità di falsi positivi dovuti a errori di misura o a movimenti transitori durante le operazioni di preparazione al decollo.

Conferma temporale del decollo : Una volta rilevato il superamento delle soglie di velocità e altitudine, il sistema non modifica immediatamente il proprio stato operativo. Viene invece avviato un intervallo di osservazione durante il quale le condizioni devono

rimanere valide in modo continuativo. Se l'intervallo di tempo quando sono state confermate le due variabili è maggiore di **TEMPO_DECOLLO_SICURO_MS** allora si ha la certezza che il drone sia decollato in sicurezza. Qualora una delle due condizioni venga meno prima del completamento del tempo di validazione, il processo viene annullato e il conteggio temporale viene azzerato. In questo modo il sistema risulta immune a:

- brevi salti delle misure altimetriche;
- errori momentanei della stima di velocità;
- vibrazioni durante la corsa di decollo;
- disturbi generati dai sensori nelle fasi iniziali.

Solo una condizione stabile e persistente viene interpretata come un reale distacco dal suolo.

4.1. Modulo di Crash Detection e Mitigazione Danni Post-Impatto (*gestisciSchianto*)

Questa funzione opera ad altissima priorità ed è incaricata del monitoraggio continuo dell'integrità strutturale del telaio. L'architettura si basa su una macchina a stati finiti con logica di *latching*. In apertura di routine vengono valutate due condizioni distinte con effetti diversi: se *schianto_sicurezza* è disattivato, il flag *statoSchiantoRilevato* viene forzato a **false** e la funzione termina senza ulteriore elaborazione; se invece *statoSchiantoRilevato* è già **true** (schianto già rilevato), la funzione non si limita a terminare, ma ribadisce attivamente a ogni ciclo il comando di gas minimo (`motore.writeMicroseconds(GAS_M` prima di uscire, mantenendo il motore inibito. Questa prassi previene tassativamente che interferenze post-impatto o manovre a terra possano riarmare accidentalmente la propulsione.

In condizioni di volo nominale, il microcontrollore interroga via I2C il Digital Signal Processor (DSP) dell'unità BNO055. Viene estratto il vettore tridimensionale dell'accelerazione lineare (epurato dal vettore gravità di $1g$), dal quale si ricava la magnitudine istantanea impressa al baricentro tramite la norma euclidea:

$$a_{tot} = \sqrt{a_x^2 + a_y^2 + a_z^2}$$

Il valore scalare ottenuto, espresso in m/s^2 , viene comparato in tempo reale con la soglia critica di tolleranza pre-impostata a **SOGLIA_G_SCHIANTO** = $50 m/s^2$ (pari a circa $5.1g$ netti). Una sollecitazione che superi tale limite meccanico è ingegneristicamente incompatibile con le normali dinamiche del volo ad ala fissa, indicando un possibile urto balistico contro il terreno o un ostacolo.

Al fine di garantire la massima affidabilità del sistema e prevenire l'innescio di falsi positivi (*Spurious Triggers*), l'algoritmo implementa un duplice livello di filtraggio logico, strutturato in una fase di validazione del decollo e in una successiva fase di conferma

dell'impatto.

In primo luogo, per evitare attivazioni accidentali durante la manipolazione a terra, il trasporto a mano o l'armamento in pista, il monitoraggio degli urti è strettamente subordinato alla variabile di stato globale `droneInVolo`, gestita dalla funzione dedicata `verifica_drone_in_volo()`. Tale variabile commuta a `true` esclusivamente quando due condizioni cinematiche risultano soddisfatte in modo simultaneo: la velocità stimata supera la `SOGLIA_VELO_DECOLLO_MS` (5.0 m/s) e l'altitudine relativa al suolo è maggiore della `SOGLIA_ALT_DECOLLO_M` (5.0 m). Affinché il decollo sia convalidato, questa condizione congiunta deve permanere ininterrottamente per un tempo minimo (`TEMPO_DECOLLO_SICURO_MS`) pari a 1.5 secondi. Un *timestamp* di riferimento (`timestampDecollo`) viene azzerato qualora una delle due metriche scenda sotto i valori limite, riavviando il conteggio della persistenza. Questo duplice requisito — congiunzione logica e persistenza temporale — rende il rilevamento del decollo robusto e insensibile a manovre manuali o rimbalzi transitori in pista.

Solo a valle della conferma dello stato di volo, l'algoritmo di rilevamento degli impatti viene armato. A questo punto interviene il secondo livello di sicurezza: per prevenire falsi positivi indotti da singoli picchi spuri di accelerazione (quali turbolenze severe o isolate vibrazioni strutturali), il superamento della soglia meccanica critica non innesca istantaneamente il protocollo di emergenza. Il firmware esige infatti un numero minimo di campioni anomali consecutivi, definito da `SEMPLI_VALORI_SCHIANTO` = 3, prima di dichiarare l'evento di impatto. Un contatore dedicato (`contatoreImpatto`) si incrementa a ogni ciclo di esecuzione in cui l'accelerazione eccede la soglia e viene immediatamente azzerato non appena una singola lettura rientra nei limiti nominali. Tale approccio assicura che esclusivamente una sollecitazione anomala e sostenuta nel tempo — e non un singolo artefatto di campionamento — possa innescare le successive procedure di mitigazione. Il raggiungimento delle 3 letture consecutive innesca istantaneamente il protocollo di *Catastrophic Failure*, innescando entro frazioni di millisecondo le seguenti contromisure di mitigazione (*Mitigation Actions*):

1. **Inibizione Propulsiva e Logica:** I flag `statoSchiantoRilevato` e `schiantoBloccato` vengono asseriti in via definitiva. Il duty-cycle verso il propulsore viene abbattuto forzatamente alla soglia passiva di $1000\mu s$.
2. **Distacco Attuatori (Hardware Detach):** Il microcontrollore revoca l'abilitazione hardware PWM ai pin dei quattro servomotori (`detach()`). Questa manovra rimuove la coppia di tenuta elettrica dei flap. Nel caso in cui il velivolo termini la sua corsa capovolto o incastrato, il distacco impedisce ai servomotori di spingere contro le resistenze al suolo, azzerando il rischio di correnti di stallo critiche, fusione del bus a 5V (BEC) e sgranamento meccanico dei pignoni.
3. **Diagnostica e Recupero:** Viene eseguita la stampa terminale della severità d'impatto (espressa in m/s^2) ai fini di un'analisi *post-crash*. Contestualmente,

l'HMI visiva viene saturata portando i LED (Rosso, Verde, Blu) a stato **HIGH**, affiancata dall'attivazione del trasduttore acustico a una frequenza penetrante di 2000 Hz. Quest'ultimo funge da vero e proprio *Search and Rescue Beacon* (faro di localizzazione), facilitando il recupero fisico dell'hardware sul terreno.

5. Flag di Sicurezza Disattivabili da Terra

Il firmware espone quattro interruttori software (flag booleani), tutti inizializzati a **true** all'avvio, che consentono di disattivare selettivamente specifici sottosistemi di sicurezza. Tale possibilità è pensata per scenari di test a banco, diagnosi in campo o gestione di guasti dei sensori stessi, e ciascun flag è telecomandabile individualmente tramite il canale uplink descritto in §2.5.

servo_sicurezza: Controlla l'esecuzione della diagnostica di corrente sui quattro servomotori (funzione `diagnosticaServ()`, §6.0.1). Se disattivato (**false**), la funzione forza immediatamente tutti i flag di salute dei servi (`estSX_Ok`, `estDX_Ok`, `intSX_Ok`, `intDX_Ok`) a **true** e termina, sospendendo qualsiasi rilevamento di anomalia di corrente.

Comandi: `SICUREZZA_SERVI_ON` / `SICUREZZA_SERVI_OFF`.

alimentazione_sicurezza: Controlla il monitoraggio delle tensioni di batteria (funzione `gestisciAlimentazione()`), attualmente non documentata in una sottosezione dedicata del presente documento. Se disattivato, i flag `batteriaBassa_teen` e `batteriaBassa_motore` vengono forzati a **false** e la funzione termina, disabilitando sia la segnalazione di batteria scarica sia l'eventuale commutazione automatica del relè di ridondanza.

Comandi: `SICUREZZA_ALIMENTAZIONE_ON` / `SICUREZZA_ALIMENTAZIONE_OFF`.

schianto_sicurezza: Controlla l'intero modulo di rilevamento collisioni (funzione `gestisciSchianto()`, §4.1). Se disattivato, il flag `statoSchiantoRilevato` viene forzato permanentemente a **false** e la funzione termina, sospendendo completamente il monitoraggio dell'accelerazione IMU e l'eventuale latching di emergenza post-impatto.

Comandi: `SICUREZZA_SCHIANTO_ON` / `SICUREZZA_SCHIANTO_OFF`.

sistema_sicurezza_temp: Controlla l'applicazione del limite termico calcolato da `gasMaxTermico()` (§7.4). Se disattivato, il valore di gas richiesto non viene più saturato al limite termico, anche qualora la temperatura del motore rientri nella zona di derating o superi la soglia critica.

Comandi: `SICUREZZA_TEMP_ON` / `SICUREZZA_TEMP_OFF`.

AVVERTENZA: La disattivazione di uno qualsiasi di questi flag rimuove una protezione attiva del sistema. In particolare, disattivare `schianto_sicurezza` durante il volo elimina l'override automatico del propulsore in caso di impat-

to rilevato dall'IMU. L'uso di questi comandi in volo operativo deve essere rigorosamente limitato a scenari diagnostici controllati.

6. Acquisizione Sensoriale Integrata e Navigazione Autonoma

Il segmento centrale del ciclo di esecuzione (`loop`) costituisce il nucleo operativo dell'algoritmo di volo. In questa fase, il microcontrollore acquisisce i dati ambientali e cinematici, diagnostica lo stato degli attuatori e calcola la traiettoria ottima per il raggiungimento dei *waypoint* programmati.

6.0.1. Telemetria Attuatori (Health Monitoring)

La funzione `diagnosticaServo()` esegue un'analisi in tempo reale dell'assorbimento elettrico delle quattro superfici di controllo. Interrogando i sensori INA219, il sistema verifica che la corrente I di ogni singolo servomotore rientri in una stretta finestra operativa di sicurezza:

$$0.5 \text{ mA} < I < 2500 \text{ mA} \quad (1)$$

Il limite inferiore (0.5 mA) rileva istantaneamente un'avaria di tipo *Open Circuit* (es. cablaggio tranciato o connettore scollegato). Il limite superiore (2500 mA) previene danni a cascata intercettando condizioni di *Short Circuit* o stallo meccanico dell'attuatore (es. cerniera bloccata o sovraccarico aerodinamico). Se una condizione non è verificata, il sistema non invalida immediatamente il servo: un contatore di errori consecutivi dedicato a ciascun attuatore viene incrementato a ogni ciclo di anomalia e azzerato non appena la corrente rientra nella finestra nominale. Solo al superamento di 5 errori consecutivi (cioè al 6° ciclo anomalo consecutivo) il relativo flag booleano di salute viene invalidato, consentendo al Mixer a valle di isolare il guasto. Questo meccanismo di debounce previene la disattivazione di un servo funzionante a causa di singoli picchi di corrente transitori (es. durante un rapido cambio di direzione della superficie di controllo).

6.1. Monitoraggio dell'Alimentazione e Failover del Relè

La funzione `gestisciAlimentazione()`, richiamata a ogni ciclo del loop principale, interroga i moduli INA219 dedicati per determinare la tensione di bus della batteria motore (`sensoreMotore.getBusVoltage_V()`) e della batteria logica Teensy (`sensoreTeensy.getBusVoltage_V()`).

Il flag `batteriaBassa_tensy` viene asserito quando la tensione Teensy scende sotto `VALORE_BATT_TEENSY_BASSA` (4.9 V). Al primo rilevamento di questa condizione (controllo sul flag `relèAttivato` per evitare riattivazioni ridondanti), il firmware attiva il relè di ridondanza (`PIN_RELE` portato a `HIGH`), commutando l'alimentazione della logica di bordo sulla batteria del motore.

Il flag `batteriaBassa_motore` viene asserito quando la tensione della batteria motore scende sotto `VALORE_BATT_MOTORE_BASSA` (13.5 V); a differenza della soglia Teensy, questa condizione **non** attiva alcuna commutazione automatica del relè, ma alimenta unicamente la segnalazione LED (§ 13.2) e il bit 1 della bitmask di allarme telemetrico (§ 15.1.2).

6.2. Cinematica e Profilo Altimetrico

L'assetto tridimensionale dell'aeromobile viene campionato prelevando le componenti vettoriali degli angoli di Eulero (Pitch, Roll, Yaw) processate internamente dal DSP del sensore inerziale BNO055. Parallelamente, il modulo barometrico fornisce la misurazione della pressione atmosferica assoluta, dalla quale viene derivata l'altitudine barometrica relativa al suolo (`G_altitudine_baro`), sottraendo la tara calcolata durante il Pre-Flight Check. Questa misura viene successivamente fusa con il sensore LIDAR TF-Luna (§6.2.1) per ottenere la quota operativa definitiva `G_altitudine`. Il codice acquisisce inoltre i parametri termici dell'avionica e del propulsore, applicando per quest'ultimo la funzione di trasferimento lineare tipica dei trasduttori a semiconduttore (es. TMP36):

$$T(^{\circ}\text{C}) = (V_{\text{out}} - 0.5) \cdot 100 \quad (2)$$

6.3. Telemetria Laser TF-Luna e Fusione con il Barometro

Per le quote di volo prossime al suolo, dove la risoluzione barometrica risulta insufficiente per manovre di precisione come l'atterraggio, il sistema integra un telemetro laser a tempo di volo TF-Luna. Il sensore comunica tramite la porta seriale hardware dedicata `Serial2`, configurata alla velocità di 115200 baud (`BAUD_RATE_LIDAR`).

Durante la fase di inizializzazione, il firmware verifica per una finestra temporale di 3 secondi la presenza di un frame valido, riconoscibile dai due byte di header consecutivi `0x59 0x59`. Se il sensore non risponde entro tale finestra, il sistema prosegue l'avvio senza LIDAR (sensore opzionale, non bloccante), impostando il flag `lidarOk` a `false`.

La funzione `aggiornaLidar()` viene richiamata ad ogni ciclo del loop principale. Il sensore viene interrogato solo quando l'altitudine barometrica corrente è inferiore alla soglia `ALTEZZA_MAX_LIDAR` (6.0 m), poiché oltre tale quota il telemetro laser perde di significato operativo (range utile limitato) e il buffer seriale viene semplicemente svuotato senza elaborazione. Quando attivo, ogni frame a 9 byte viene validato tramite checksum (somma dei primi 8 byte confrontata con il nono); la distanza, codificata su 16 bit nei byte 2-3 del pacchetto, viene convertita da centimetri a metri.

La lettura grezza viene filtrata con un filtro passa-basso del primo ordine a guadagno fisso `ALPHA_LIDAR = 0.25`:

$$G_altitudine_lidar(t) = 0.25 \cdot \text{distanza_m} + 0.75 \cdot G_altitudine_lidar(t - 1) \quad (3)$$

Fusione con il barometro. La variabile operativa di quota `G_altitudine`, utilizzata da tutti i loop di controllo e dalla navigazione, viene determinata dalla funzione `aggiorna_altitudine()` secondo la seguente logica di selezione prioritaria: se il LIDAR è operativo (`lidarOk`), fornisce una lettura valida (`G_altitudine_lidar > 0`) e la quota barometrica è inferiore alla soglia `ALTEZZA_MAX_LIDAR` (6.0 m), il sistema adotta la quota LIDAR come riferimento principale, per la sua maggiore risoluzione e assenza di deriva in prossimità del suolo. In tutti gli altri casi (quota elevata, LIDAR assente o non ancora inizializzato), il sistema ricade sulla quota barometrica `G_altitudine_baro`.

Questa architettura di fusione a commutazione (anziché a media pesata continua) garantisce continuità di misura anche in assenza del sensore laser, mantenendo al contempo la precisione centimetrica necessaria nelle fasi di avvicinamento al suolo.

6.4. Anemometria, Cinematica GPS e Sensor Fusion

La determinazione del vettore velocità dell'aeromobile rappresenta un parametro di importanza vitale sia per il sostentamento aerodinamico (prevenzione dello stallo) sia per la navigazione tattica. A tale scopo, il firmware implementa un'architettura di *Sensor Fusion* che accoppia la fluidodinamica classica alla radionavigazione satellitare.

La stima della velocità rispetto all'aria (*True Airspeed*, TAS) è affidata a un tubo di Pitot connesso a un trasduttore di pressione differenziale analogico. Il convertitore ADC a 10 bit del microcontrollore campiona il livello di tensione, dal quale viene sottratto l'offset di zero (`VALORE_ZERO`) pre-calibrato al suolo. Il dato netto viene vincolato a valori non negativi tramite una funzione di *clamping* (`constrain`) per garantire la stabilità nel dominio reale delle successive radici quadrate.

La differenza analogica viene scalata in Pascal (ΔP) e utilizzata per ricavare la velocità aerodinamica istantanea V_{pitot} , invertendo l'equazione di Bernoulli per i fluidi incomprimibili:

$$V_{pitot} = \sqrt{\frac{2\Delta P}{\rho}} \quad (4)$$

A differenza di una densità dell'aria fissata a condizioni standard, il firmware ricalcola ρ a ogni ciclo del loop principale tramite la funzione `aggiorna_densita_aria()`, applicando l'equazione di stato dei gas perfetti alla pressione statica e alla temperatura misurate dal barometro BMP390:

$$\rho = \frac{P}{R_{specifico} \cdot T} \quad (5)$$

dove P è la pressione assoluta letta dal barometro (Pa), T la temperatura assoluta in Kelvin (temperatura barometrica +273.15) e $R_{specifico} = 287.05 \text{ J}/(\text{kg} \cdot \text{K})$ la costante specifica dell'aria secca. Il valore $1.225 \text{ kg}/\text{m}^3$ è utilizzato unicamente come inizializzazione di sicurezza prima della prima lettura barometrica valida.

Parallelamente all'anemometria, il modulo `TinyGPSPlus` estrapola la velocità cinematica

rispetto al suolo (*Ground Speed*, GS) tramite il metodo `gps.speed.mps()`.

Il firmware **non** implementa un filtro complementare pesato tra Pitot e GPS: le due grandezze sono utilizzate per scopi distinti. La velocità all'aria impiegata dai loop PID (`G_Airspeed_ms`) deriva primariamente dal Pitot, ritenuto valido solo se strettamente positivo e inferiore alla soglia `MAX_AIRSPEED_X8` (45.0 km/h, pari a 12.5 m/s); in assenza di lettura Pitot valida, il sistema ricade sulla velocità al suolo (`G_Groundspeed_ms`) purché il GPS sia valido oppure il velivolo si trovi sotto la quota `ALTEZZA_MAX_SENSORE_OTTICO` (4.0 m), altrimenti l'airspeed viene azzerata. La velocità al suolo, utilizzata invece dalla legge di guidance L1 (§ 6.6), è calcolata separatamente dalla funzione `aggiorna_velocita()`: sotto i 4 m di quota viene privilegiata la stima del sensore di flusso ottico PMW3901 quando disponibile, altrimenti si utilizza direttamente la velocità GPS.

6.5. Flusso Ottico PMW3901 — Stima di Velocità a Bassa Quota

A integrazione della sensor fusion Pitot/GPS descritta in §6.3, il sistema impiega un sensore di flusso ottico Bitcraze PMW3901, collegato tramite interfaccia SPI con pin di selezione (Chip Select) sul pin 25 del microcontrollore.

Il sensore viene interrogato dalla funzione `velocità_flusso_ottico()`, che legge ad ogni ciclo il conteggio di moto orizzontale (dx , dy) tramite il metodo `readMotionCount()`. Poiché il PMW3901 misura uno spostamento angolare del pattern ottico sottostante, la conversione in velocità lineare richiede la conoscenza della quota corrente: la costante di scala `COSTANTE_OTTICA` (0.0012) e l'altitudine attuale `G_altitudine` vengono impiegate per ricavare le componenti di velocità nel sistema di riferimento del sensore:

$$v_x = \frac{dx \cdot \text{COSTANTE_OTTICA} \cdot G_altitudine}{\Delta t} \quad (6)$$

$$v_y = \frac{dy \cdot \text{COSTANTE_OTTICA} \cdot G_altitudine}{\Delta t} \quad (7)$$

Tali componenti vengono poi ruotate nel sistema di riferimento terrestre tramite l'angolo di imbardata (*yaw*) corrente, ottenendo le velocità globali `G_vel_x_optical_sensor` e `G_vel_y_optical_sensor`.

Condizione di validità — `ALTEZZA_MAX_SENSORE_OTTICO`. Il sensore di flusso ottico è per sua natura affidabile solo a bassa quota, dove il pattern del terreno rimane ben risolto otticamente. Il firmware vincola pertanto l'utilizzo della misura alla condizione `G_altitudine < ALTEZZA_MAX_SENSORE_OTTICO` (4.0 m). Oltre questa soglia, oppure in caso di Δt non valido, le variabili di velocità ottica vengono invalidate impostandole a -1.0 come valore sentinella.

Contributo alla sensor fusion e relazione con il GPS. All'interno della funzione `aggiorna_velocita()`, la velocità al suolo `G_Groundspeed_ms` viene determinata tramite una logica di selezione a priorità (non un filtro complementare pesato): quando il drone si

trova sotto la soglia `ALTEZZA_MAX_SENSORE_OTTICO` (4.0 m) e le letture del flusso ottico sono valide, il sistema calcola la magnitudine del vettore planare rilevato otticamente:

$$G_Groundspeed_ms = \sqrt{G_vel_x_optical_sensor_ms^2 + G_vel_y_optical_sensor_ms^2} \quad (8)$$

In tutti gli altri casi (quota superiore alla soglia, oppure lettura ottica non valida), `G_Groundspeed_ms` viene impostata direttamente pari alla velocità GPS (`gps.speed.mps()`), se disponibile, oppure a zero.

Questo comportamento riflette una gerarchia di affidabilità dipendente dalla quota. Si noti che `G_Groundspeed_ms` alimenta esclusivamente la logica di navigazione L1 (§ 6.6); la velocità utilizzata dal loop PID di Autothrottle (§ 12.2.5) è invece `G_Airspeed_ms`, basata primariamente sul Pitot come descritto in § 6.4.

6.6. Algoritmo di Navigazione Non-Lineare (L1 Guidance Law)

La funzione `aggiornaNavigazione()` rappresenta l'apice dell'autonomia tattica del sistema. Essa sfrutta la libreria `TinyGPSPlus` per calcolare la distanza geodetica (formula dell'Haversine) e la rotta ortodromica verso il target (`TARGET_LAT`, `TARGET_LON`).

Prima di calcolare la guida L1, la funzione `aggiornaNavigazione()` verifica il raggiungimento del waypoint corrente confrontando la distanza dal target con un raggio di accettazione dinamico. Tale raggio non coincide semplicemente con il valore minimo configurabile `RAGGIO_ACCETTAZIONE_MINIMO_m` (25.0 m di default, modificabile da terra con il comando `SET_RAGGIO_WAYPOINT`), ma viene scalato in funzione della velocità corrente secondo la relazione:

$$raggio_dinamico = \max(RAGGIO_ACCETTAZIONE_MINIMO_m, 0.75 \cdot L_1) \quad (9)$$

dove L_1 è la medesima distanza di look-ahead utilizzata dalla legge di guida descritta di seguito. Se la distanza dal target è inferiore o uguale al raggio dinamico, il waypoint viene considerato raggiunto: la distanza residua viene azzerata e la funzione termina senza calcolare un nuovo comando di rollo, in attesa dell'assegnazione del target successivo.

Per guidare l'aeromobile sulla traiettoria desiderata evitando sovraelongazioni (*overshoot*), viene implementata la logica di guida non-lineare **L1**. L'algoritmo definisce innanzitutto una distanza di *look-ahead* L_1 , proporzionale alla velocità istantanea dell'aeromobile moltiplicata per una costante di smorzamento (nel codice: 4.0 secondi). La velocità stimata viene preventivamente vincolata a un minimo di 1.0 m/s per evitare singolarità matematiche e divisioni per zero.

Sia η l'errore di rotta normalizzato nell'intervallo $[-\pi, \pi]$ (differenza tra la rotta verso il target e la direzione di moto attuale). Per velocità rispetto al suolo superiori a 2.0 m/s, il sistema privilegia la rotta calcolata dal GPS; al di sotto di tale soglia, o in assenza di rotta

GPS valida, il calcolo ricade sullo Yaw attuale fornito dalla bussola IMU. L'accelerazione laterale a_{lat} richiesta per curvare verso il punto L1 è definita dall'equazione geometrica:

$$a_{lat} = 2 \frac{V^2}{L_1} \sin(\eta) \quad (10)$$

Tale accelerazione centripeta viene generata inclinando l'ala fissa. Il controllore calcola l'angolo di rollio bersaglio (ϕ_{target}) bilanciando la portanza inclinata con l'accelerazione di gravità g :

$$\phi_{target} = \arctan\left(\frac{a_{lat}}{g}\right) \quad (11)$$

L'angolo risultante viene convertito in gradi e sottoposto a *clamping* (**constrain**) in base al limite meccanico di rollio consentito (**MAX_ROLL**), fornendo un setpoint stabile e aerodinamicamente sicuro ai loop del controllore PID a valle.

7. Preparazione Adattiva del Profilo Propulsivo

7.1. Logica di Selezione della Fase di Volo

Prima di qualsiasi interazione con la radio o i controllori PID, il firmware determina il **regime propulsivo obiettivo** confrontando la distanza attuale dal waypoint di destinazione — calcolata al ciclo precedente dalla funzione `aggiornaNavigazione()` e memorizzata nella variabile globale `G_distanzaDalTarget_m` — con la soglia di commutazione `DISTANZA_FRENATA`.

La discriminante è la seguente:

$$\text{Fase} = \begin{cases} \text{Crociera} & \text{se } d_{target} > 150 \text{ m} \\ \text{Avvicinamento} & \text{se } d_{target} \leq 150 \text{ m} \end{cases}$$

7.2. Fase di Crociera

Quando la distanza dal waypoint supera i **150 m** (`DISTANZA_FRENATA`), il sistema si trova in regime di crociera: il controllore di velocità riceve come riferimento `targetVelocita` = 60.0 km/h (`VELOCITA_CROCIERA`), mentre il valore di gas di base trasmesso al PID propulsivo è fissato a `gasDiBase` = 1450 μ s (`GAS_CROCIERA`). Quest'ultimo rappresenta la posizione angolare nominale dell'ESC in regime stazionario, attorno alla quale il PID di velocità opera le proprie correzioni incrementali.

7.3. Fase di Avvicinamento Finale

All'ingresso nella **bolla di avvicinamento** (distanza dal target ≤ 150 m), il firmware riduce automaticamente i riferimenti propulsivi: la velocità obiettivo scende a **45 km/h** (VELOCITA_AVVICINAMENTO) e il gas di base viene abbassato a **1250 μ s** di posizione ESC (valore fisso, non associato a una costante nominata). La decelerazione ha lo scopo di contenere l'energia cinetica residua del velivolo nella fase terminale della missione, agevolando una traiettoria di atterraggio o di sgancio payload più controllata.

Fase	Distanza soglia	Velocità target	Gas di base (ESC)
Crociera	> 150 m	60.0 km/h	1450 μ s
Avvicinamento	≤ 150 m	45.0 km/h	1250 μ s

Nota Tecnica

I valori `targetVelocita` e `gasDiBase` non agiscono direttamente sull'ESC: fungono da *setpoint* e *punto di lavoro* per il controllore PID di velocità, che li elabora nel ciclo successivo generando il comando gas finale `comandoGasFinale`. La separazione tra riferimento e attuazione è intenzionale e garantisce la continuità del controllo in retroazione.

7.4. Limitazione Termica del Gas (`gasMaxTermico`)

Indipendentemente dal regime di volo selezionato (Crociera o Avvicinamento, cfr. §7.2–7.3) o dalla modalità operativa (Manuale/Auto), il firmware applica un ulteriore livello di protezione a tutela del gruppo propulsore, basato sulla temperatura del motore `Global_Temperatura_motore` (§2.1).

La funzione `gasMaxTermico()` calcola dinamicamente il valore massimo di gas erogabile in funzione della temperatura corrente, impostando una curva di derating a tre tratti:

- **Temperatura $\leq 70^\circ\text{C}$ (`T_MOTORE_THROTTLE_START`):** nessuna limitazione; il gas massimo consentito coincide con `GAS_MASSIMO` (2000 μ s).
- **Temperatura $\geq 90^\circ\text{C}$ (`T_MOTORE_THROTTLE_END`):** limitazione massima; il gas viene ridotto al valore `GAS_MINIMO` (1200 μ s), indipendentemente dalla richiesta del pilota o del PID.
- **Temperatura compresa tra 70°C e 90°C :** interpolazione lineare tra i due estremi, determinata dal fattore adimensionale t :

$$t = \frac{T - 70}{90 - 70} \quad (12)$$

$$\text{limite} = \text{GAS_MASSIMO} - t \cdot (\text{GAS_MASSIMO} - \text{GAS_MINIMO}) \quad (13)$$

Il valore così calcolato viene confrontato, a ogni ciclo del loop principale, con il comando di gas finale (`comandoGasFinale`): se quest'ultimo eccede il limite termico e il flag `sistema_sicurezza_temp` è attivo, il comando viene saturato al valore limite prima di essere inviato all'ESC. La protezione agisce quindi come un tetto dinamico sovrapposto a qualsiasi altra logica di controllo (manuale, crociera, avvicinamento o PID *autothrottle*), prevenendo il surriscaldamento del propulsore in condizioni di richiesta di potenza prolungata.

8. Lettura del Ricevente S.BUS e Selezione della Modalità

8.1. Acquisizione del Frame S.BUS

Il ricevente FrSky trasmette in modo continuo pacchetti S.BUS sul bus seriale `Serial7`. La libreria `SBUS` decodifica questi frame e ne espone il contenuto tramite il metodo `read()`, che popola:

- L'array `canaliRC[16]`: valori grezzi dei 16 canali RC nell'intervallo tipico [172, 1811], corrispondenti rispettivamente allo stick al minimo e allo stick al massimo.
- Il flag booleano `failsafe`: attivato dal ricevente stesso qualora non riceva segnale valido dal trasmettitore per un periodo superiore alla soglia configurata.
- Il flag `pacchettoPerso`: segnala la perdita di singoli frame S.BUS, distinguibile dalla condizione di failsafe prolungato.

La chiamata a `read()` restituisce `true` solo in presenza di un nuovo frame valido e completamente decodificato, pertanto l'aggiornamento della modalità avviene esclusivamente in corrispondenza di dati freschi, evitando decisioni basate su campioni obsoleti.

8.2. Decodifica del Canale di Selezione Modalità

Il **canale 5** del radiocomando (indice `canaliRC[4]` nell'array zero-based) è dedicato alla selezione della modalità di volo. Si tratta tipicamente di un interruttore a due o tre posizioni sul trasmettitore, il cui valore grezzo viene confrontato con la soglia di **992 unità S.BUS**:

$$\text{global_modalitaVolo} = \begin{cases} 1 \text{ (Manuale)} & \text{se } \text{canaliRC}[4] < 992 \\ 2 \text{ (Auto GPS)} & \text{se } \text{canaliRC}[4] \geq 992 \text{ e } \text{droneInVolo} = \text{true} \text{ e } \text{statoSch} \\ \text{(invariato)} & \text{se } \text{canaliRC}[4] \geq 992 \text{ ma le condizioni precedenti non sono} \end{cases}$$

Il valore 992 corrisponde alla posizione centrale del range S.BUS [172, 1811], fungendo da soglia di bisezione netta tra i due stati dell'interruttore. Quando `canaliRC[4] ≥`

992, il passaggio a modalità 2 (Auto GPS) è quindi condizionato al fatto che il drone sia effettivamente in volo e che non sia stato rilevato uno schianto: se una delle due condizioni non è verificata, `global_modalitaVolo` rimane invariato rispetto al ciclo precedente. Questa condizione aggiuntiva è coerente con quanto rappresentato nel diagramma a stati di Figura 1 (§9).

8.3. Arbitraggio Finale: Failsafe come Override Prioritario

Indipendentemente dalla selezione manuale del pilota, il **failsafe sovrascrive qualsiasi modalità** con priorità assoluta. La variabile `statoAttuale` — distinta dalla `global_modalitaVolo` per preservare quest'ultima intatta per logiche successive — viene impostata a **3** in presenza di failsafe, a prescindere dal valore letto sul canale 5.

$$\text{statoAttuale} = \begin{cases} 3 & \text{se failsafe} = \text{true} \\ \text{global_modalitaVolo} & \text{altrimenti} \end{cases}$$

Avvertenza

La separazione logica tra `global_modalitaVolo` e `statoAttuale` è una scelta architetturale deliberata: in condizione di failsafe, il sistema deve comportarsi come in modalità Auto (ricorrendo ai PID per il rientro autonomo), ma la variabile di modalità non deve essere alterata, affinché al ripristino del segnale radio il sistema torni allo stato selezionato dal pilota senza richiedere un'azione esplicita.

9. Macchina a Stati della Modalità di Volo

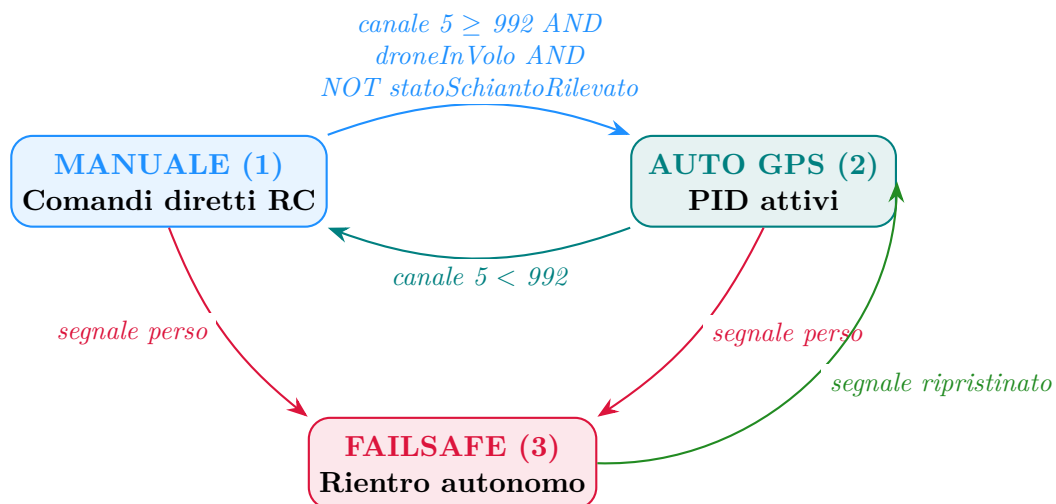


Figura 1: Diagramma di transizione della macchina a stati della modalità di volo. Le frecce in rosso indicano transizioni forzate dal failsafe, indipendenti dalla volontà del pilota.

10. Reset dei Controllori PID al Cambio di Modalità

10.1. Rilevamento della Transizione di Stato

Il firmware mantiene una variabile statica locale `modalitaPrecedente` — inizializzata a 1 (Manuale) — che viene confrontata ad ogni ciclo con `statoAttuale`. Una **discrepanza tra i due valori** segnala una transizione di modalità appena avvenuta, innescando la sequenza di reset.

L'utilizzo della qualifica `static` per questa variabile è essenziale: garantisce la persistenza del valore tra cicli successivi del loop, comportandosi di fatto come una variabile globale con scope locale alla funzione.

10.2. Procedura di Reset delle Variabili di Stato PID

In occasione di ogni transizione di modalità, **tutte le variabili di stato** dei quattro controllori PID vengono azzerate simultaneamente:

Variabile azzerata	Tipo	Controllore
<code>pid_sommaErroriAlt</code>	Integrale	Altitudine
<code>pid_errorePassatoAlt</code>	Derivativo	Altitudine
<code>pid_sommaErroriPitch</code>	Integrale	Beccheggio
<code>pid_errorePassatoPitch</code>	Derivativo	Beccheggio
<code>pid_sommaErroriRoll</code>	Integrale	Rollio
<code>pid_errorePassatoRoll</code>	Derivativo	Rollio
<code>pid_sommaErroriVel</code>	Integrale	Velocità
<code>pid_errorePassatoVel</code>	Derivativo	Velocità

Contestualmente, il riferimento temporale `tempoPassatoPID` viene aggiornato con il timestamp corrente tramite `millis()`, azzerando la storia temporale del controllore e impedendo che un Δt anomalo — conseguente alla pausa tra le modalità — generi un picco derivativo (*derivative kick*) al primo ciclo successivo alla transizione.

Nota Tecnica

L'azzeramento degli accumulatori integrali alla transizione di modalità è una pratica fondamentale nella progettazione di sistemi di controllo ibridi. In assenza di tale reset, la sommatoria accumulata nella modalità precedente — potenzialmente relativa a un assetto completamente diverso — verrebbe iniettata come disturbo all'ingresso degli attuatori, causando manovre violente e non intenzionali nei primi istanti della nuova

modalità.

10.3. Sblocco di Emergenza Post-Schianto

Lo sblocco di emergenza **non** è legato alla rilevazione della transizione di modalità descritta in precedenza (confronto `statoAttuale/modalitaPrecedente`), bensì è un controllo indipendente eseguito all'interno del blocco di lettura S.BUS: ad ogni nuovo frame valido (`ricevente.read(...)`), se `statoSchiantoRilevato` è attivo e il canale 5 è riportato in posizione Manuale (`canaliRC[4] < 992`), il firmware interpreta l'azione del pilota come **segnale di sblocco intenzionale** e procede con:

1. Azzeramento dei flag `statoSchiantoRilevato` e `schiantoBloccato`, riportando il sistema in uno stato di operatività normale.
2. Azzeramento del flag `droneInVolo`, resettando la logica di rilevamento del volo attivo.
3. Riesecuzione di `inizializzazione_servo()`: i quattro servomotori vengono ricollegati (`attach()`) e riportati alla posizione neutra `CENTRO_SERVO`, ripristinando anche i flag `statoPrecedenteInterni` e `statoPrecedenteEsterni` a `true`.
4. Silenziamento del buzzer di allarme (`noTone()`), che durante lo schianto emette un segnale continuo di soccorso.
5. Emissione di una sequenza sonora di conferma a due toni (1000 Hz poi 1500 Hz, 100 ms ciascuno).
6. Stampa di un messaggio diagnostico sulla porta seriale: *«!!! SBLOCCO EMERGENZA ESEGUITO DA RADIO !!! Servi Riarmati e centrati.»*

Avvertenza

Questo meccanismo di sblocco è deliberatamente vincolato a un'azione *esplicita* del pilota sul radiocomando: non è automatico né temporizzato. Ciò garantisce che il sistema non riprenda autonomamente l'operatività dopo un impatto senza la supervisione diretta dell'operatore in campo.

11. Modalità Manuale — Mappatura Diretta dei Canali RC

11.1. Condizione di Attivazione

L'attivazione di questo ramo è in ogni caso subordinata alla condizione `!schiantoBloccato`: qualora sia stato rilevato un impatto (§4.1) e il flag di blocco sia attivo, nessun comando di pitch/roll/gas viene prodotto, indipendentemente dallo stato del canale 5 o del failsafe. La modalità manuale è attiva quando `global_modalitaVolo` è uguale a 1 e il flag `failsafe`

è **false**. La congiunzione di entrambe le condizioni è imprescindibile: anche con il canale 5 in posizione manuale, il failsafe ha comunque la precedenza, escludendo questa modalità e delegando il controllo ai PID.

11.2. Mappatura dei Canali RC sugli Attuatori

In modalità manuale, i valori grezzi S.BUS dei tre canali primari di pilotaggio vengono convertiti nei rispettivi comandi agli attuatori mediante la funzione `map()` di Arduino, seguita da un `constrain()` che recide eventuali valori fuori range:

11.2.1. Gas (Canale 3 — `canaliRC[2]`)

Il canale del gas viene mappato linearmente dall'intervallo grezzo S.BUS [172, 1811] all'intervallo operativo del segnale PWM inviato all'ESC, espresso in microsecondi (μs) tramite il metodo `motore.writeMicroseconds()`:

GAS_NEUTRO / GAS_MINIMO / GAS_MASSIMO — $1000 \mu s$ / $1200 \mu s$ / $2000 \mu s$ — Valori operativi del segnale PWM motore (`writeMicroseconds`): **GAS_NEUTRO** è l'idle da stick manuale, **GAS_MINIMO** il minimo operativo in volo autonomo, **GAS_MASSIMO** il pieno regime.

dove **GAS_NEUTRO** = $1000 \mu s$ rappresenta il minimo comandabile dallo stick manuale (motore fermo/idle) e **GAS_MASSIMO** = $2000 \mu s$ il pieno regime, in accordo con lo standard PWM per ESC (intervallo 1000–2000 μs). Si noti che il valore **GAS_MINIMO** = $1200 \mu s$, distinto da **GAS_NEUTRO**, rappresenta invece il minimo operativo consentito durante il volo autonomo (PID), impiegato ad esempio nelle emergenze di quota (§12.2.3) e nella limitazione termica (§7.4): la mappatura manuale utilizza deliberatamente **GAS_NEUTRO** come estremo inferiore per consentire l'arresto completo del motore da stick in modalità Manuale.

GAS_NEUTRO / MINIMO / MASSIMO ($1000 \mu s$ / $1200 \mu s$ / $2000 \mu s$) — Valori operativi del segnale PWM motore (`writeMicroseconds`): **GAS_NEUTRO** è l'idle da stick manuale, **GAS_MINIMO** il minimo operativo in volo autonomo, **GAS_MASSIMO** il pieno regime.

11.2.2. Rollio (Canale 1 — `canaliRC[0]`)

Il canale di rollio viene mappato sull'intervallo simmetrico $[-35\checkmark, +35\checkmark]$ (**MAX_ROLL**), con lo zero che corrisponde allo stick centrato (posizione neutra delle superfici):

$$\text{correzioneRoll} = \text{clip}\left(\frac{(\text{canaliRC}[0] - 172)}{(1811 - 172)} \cdot 70 - 35, -35, 35\right)$$

11.2.3. Beccheggio (Canale 2 — `canaliRC[1]`)

Il canale di beccheggio viene mappato sull'intervallo $[-20\checkmark, +20\checkmark]$ (**MAX_PITCH**). Si noti che la mappatura è **invertita** rispetto al gas e al rollio: il valore minimo S.BUS (172)

corrisponde a `MAX_PITCH` e il valore massimo S.BUS (1811) corrisponde a `-MAX_PITCH`. Questa inversione riflette la convenzione aerodinamica standard del radiocomando in modalità *Mode 2*, per cui lo stick del beccheggio tirato verso il basso (*pull*) corrisponde a un angolo positivo (muso su):

$$\text{correzionePitch} = \text{clip}\left(\frac{(\text{canaliRC}[1] - 172)}{(1811 - 172)} \cdot (-40) + 20, -20, 20\right)$$

Canale	Indice	Range S.BUS	Output	Range output
Gas	[2]	[172, 1811]	<code>comandoGasFinale</code>	[1000 _{ts} , 2000 _{ts}]
Rollio	[0]	[172, 1811]	<code>correzioneRoll</code>	[-35 _ř , +35 _ř]
Beccheggio	[1]	[172, 1811]	<code>correzionePitch</code>	[+20 _ř , -20 _ř] (inv.)

11.3. Ottimizzazione dell'Output Seriale

Per evitare la saturazione della porta seriale di debug con messaggi ridondanti — fenomeno particolarmente penalizzante in sistemi real-time dove la UART introduce latenza — il messaggio di stato «*Volo: MANUALE*» viene stampato **una sola volta per ogni transizione** verso la modalità manuale, confrontando `statoAttuale` con la variabile statica `ultimoStatoStampato`. La stampa avviene esclusivamente quando i due valori divergono, dopodiché `ultimoStatoStampato` viene aggiornato per sopprimere le stampe successive.

12. Modalità Automatica e Failsafe — Controllo PID

12.1. Condizione di Attivazione

L'attivazione dei rami Manuale e Auto/Failsafe è in ogni caso subordinata alla condizione `!schiantoBloccato`: qualora sia stato rilevato un impatto (§4.1) e il flag di blocco sia attivo, nessuno dei due rami produce comandi di pitch/roll/gas, indipendentemente dallo stato del canale 5 o del failsafe. Questa modalità è attiva in due scenari distinti, trattati dal firmware in modo funzionalmente identico seppur semanticamente differenziato:

- **Auto GPS (modalità 2):** il pilota ha selezionato esplicitamente il controllo automatico tramite l'interruttore sul trasmettitore (canale 5 $\geq$ 992). Il drone naviga autonomamente verso il waypoint con i PID attivi.
- **Failsafe (modalità 3):** il collegamento radio è stato perso. Il sistema attiva automaticamente i PID per portare il velivolo verso le coordinate target (`TARGET_LAT`, `TARGET_LON`) alla quota di sicurezza definita dalla variabile globale `ALTITUDINE_TARGET_g`

(inizializzata a **40 m AGL**, ma sovrascrivibile dinamicamente in volo tramite il comando da terra `SET_ALTITUDE`, § 2.2), presumibilmente corrispondente al punto di decollo o a una zona sicura predefinita.

Nota Tecnica

La scelta di trattare failsafe e Auto GPS con la stessa logica di controllo non è una semplificazione, ma una precisa scelta di progetto: in entrambi i casi il velivolo deve raggiungere un punto geografico noto in modo autonomo, e i controllori PID sono lo strumento adeguato per farlo.

12.2. Controllo di Volo Autonomo: Architettura PID a Cascata

Il cuore pulsante della stabilizzazione del velivolo e della navigazione autonoma è demandato alla funzione `calcolaPID()`. Questa routine implementa un'architettura di controllo proporzionale-integrale-derivativo (PID) di tipo Multi-Input Multi-Output (MIMO). Per disaccoppiare la dinamica longitudinale da quella laterale, il sistema utilizza quattro anelli di retroazione (*feedback loops*) calcolati in sequenza ad ogni iterazione del loop principale.

12.2.1. Gestione del Dominio Temporale (*Time-Step Protection*)

Per garantire che le componenti integrali e derivate dell'equazione PID rimangano matematicamente coerenti, la funzione calcola il tempo trascorso (`dt`) dall'ultima iterazione, espresso in secondi. Per prevenire divergenze dell'algoritmo (fenomeno del *Windup*), il firmware applica una doppia protezione hardware-software:

- `dt <= 0.001 s`: Evita eccezioni matematiche di divisione per zero in caso di loop eseguiti troppo rapidamente.
- `dt > 0.5 s`: Evita picchi brutali della componente derivativa in caso di latenze impreviste sul bus I2C, "clippando" il passo temporale a un massimo di mezzo secondo.

12.2.2. Parametri Operativi del Controllore

La seguente tabella riassume i guadagni di sintonizzazione (*Tuning Gains*) e i limiti meccanici operativi imposti per confinare l'involuppo di volo.

Asse di Controllo	K_p (Prop)	K_i (Int)	K_d (Deriv)
Altitudine (<code>_alt</code>)	0.5	0.05	0.2
Beccheggio (<code>_pitch</code>)	1.2	0.05	0.5
Rollio (<code>_roll</code>)	1.2	0.05	0.5
Autothrottle (<code>_vel</code>)	1.5	0.1	0.5

Costante di Vincolo	Valore	Descrizione
ALTEZZA_MAX / MIN	120 m/10 m	Quota di inviluppo (<i>Hard Deck</i> e <i>Ceiling</i>)
MAX_PITCH / ROLL	20°/35°	Limite aerodinamico strutturale
GAS_MINIMO / MASSIMO	1200 μ s/2000 μ s	Valori operativi di segnale PWM motore (autonomo)
GAS_NEUTRO	1000 μ s	Idle motore in modalità Manuale
GAS_CROCIERA	1450 μ s	Punto di lavoro nominale in fase di crociera (§7.2)

12.2.3. Loop 1: Altitudine e Pitch-Targeting

L'anello di controllo altimetrico è il loop più esterno (*Outer Loop*) della dinamica longitudinale. L'errore viene calcolato come scostamento tra l'altitudine geodetica desiderata e l'AGL attuale ricavato dal barometro. Invece di comandare direttamente le superfici di volo, l'uscita di questo PID genera un angolo di beccheggio bersaglio (`targetPitch_Auto`).

Prima dell'elaborazione matematica, il sistema esegue un controllo rigoroso sui limiti dell'infrastruttura aerea (*Hard Bounds*):

- Se Quota > 120m, il PID viene escluso. Viene forzato un beccheggio di picchiata di -8° e la base del gas viene ridotta a `GAS_MINIMO` (1200 μ s) per prevenire il superamento della VNE (*Velocity Never Exceed*).
- Se Quota < 10m, il velivolo forza un assetto di cabrata d'emergenza di $+12^\circ$ e innalza il gas a `GAS_MASSIMO` - 10 per evitare il CFIT (*Controlled Flight Into Terrain*).

L'integratore di altitudine è protetto da un vincolo di *Anti-Windup* confinato tra ± 20.0 . A valle del calcolo, la somma proporzionale, integrale e derivativa subisce una saturazione asimmetrica per calcolare l'angolo di beccheggio bersaglio (`targetPitch_Auto_g`): l'output viene forzatamente confinato nell'intervallo $[-10.0^\circ, +15.0^\circ]$ prima di essere inviato al loop di assetto interno.

12.2.4. Loop 2 e 3: Dinamica d'Assetto (Pitch e Roll)

Gli anelli interni (*Inner Loops*) di beccheggio e rollio elaborano l'errore tra i target (richiesti rispettivamente dal PID di Altitudine e dall'algoritmo di Navigazione L1) e gli angoli reali istantanei forniti dall'IMU BNO055.

Sia $e(t)$ l'errore angolare al tempo t , l'equazione implementata in tempo discreto per ogni asse è:

$$Output(t) = K_p \cdot e(t) + K_i \sum_0^t e(t) \Delta t + K_d \frac{e(t) - e(t-1)}{\Delta t} \quad (14)$$

Anche in queste sezioni, gli integratori sono confinati all'interno di un intervallo di sicurezza (± 40.0). L'output finale di questi algoritmi, destinato alla matrice di mixing, subisce un

clipping finale per impedire al drone di comandare i servomotori oltre i limiti di progetto (MAX_PITCH e MAX_ROLL).

12.2.5. Loop 4: Autothrottle e Gestione dell'Energia Cinetica

La regolazione della trazione è sganciata dalla dinamica di assetto, realizzando un vero e proprio Autothrottle aeronautico. L'algoritmo confronta la `targetVelocita` (impostata dalla navigazione a 60 km/h in rotta e a 45 km/h in fase di avvicinamento al target) con la `velocitaAttuale`, frutto della *Sensor Fusion* tra GPS e Tubo di Pitot.

Analogamente agli altri tre anelli di controllo, anche l'accumulatore integrale di questo loop è protetto da un vincolo di *Anti-Windup*, confinato nell'intervallo ± 30.0 .

Il calcolo della spinta è strutturato secondo la logica *Feed-Forward* + PID. Alla base del calcolo non si parte da zero, ma da un valore di `gasCorrente`, dettato in tempo reale dall'anello dell'altitudine (che può tagliare o aumentare il gas in caso di picchiata/cabrata d'emergenza). A questo valore di base viene sommato algebricamente l'output correttivo del PID di velocità. Il risultato è infine confinato nei limiti costruttivi del controllore ESC (`GAS_MINIMO` ... `GAS_MASSIMO`) ed erogato all'attuatore.

13. Attuazione Fisica, Override di Sicurezza e Watchdog Hardware

A valle di tutti i calcoli cinematici e della risoluzione delle matrici PID, il ciclo di esecuzione del Flight Controller converge verso la fase di attuazione fisica (*Actuation Phase*). In questo blocco critico, il sistema funge da arbitro finale (*Multiplexer* di sicurezza), decidendo se instradare i comandi di volo verso le superfici aerodinamiche o se interdire completamente il velivolo per motivi di emergenza.

13.1. Logica di Attuazione e Inibizione Motore (Crash Override)

Prima di autorizzare la scrittura dei segnali PWM (Pulse Width Modulation) sui pin di uscita, il firmware esegue un controllo assoluto e bloccante sulla variabile di stato globale `statoSchiantoRilevato`.

Qualora l'IMU abbia registrato 3 letture consecutive di accelerazione superiori alla soglia di guardia `SOGLIA_G_SCHIANTO` (50 m/s², cfr. § 5), il sistema innesca un *Hardware Override*. In questa condizione, l'algoritmo ignora deliberatamente qualsiasi output calcolato dal PID o richiesto dal pilota, e inietta il valore di idle `motore.writeMicroseconds(GAS_NEUTRO)` all'Electronic Speed Controller (ESC), anziché un valore nullo. L'utilizzo di `GAS_NEUTRO` (1000 μ s), anziché di un impulso PWM pari a zero, garantisce che l'ESC riceva comunque un segnale PWM valido e riconosciuto come comando di minimo, evitando che il controller entri

in uno stato di segnale assente (che su alcuni ESC può generare allarmi o comportamenti indefiniti). Questa inibizione totale azzerla la spinta, mettendo in sicurezza la zona d'impatto ed evitando l'autodistruzione del propulsore contro eventuali ostacoli a terra.

Viceversa, se l'aeromobile opera in condizioni nominali (nessuna collisione rilevata), il flusso logico entra nel ramo condizionale standard. Solo ed esclusivamente in questo caso vengono inviati i comandi di attuazione:

1. Viene richiamata la routine di interfaccia `gestisci_allarmi()` per aggiornare il pannello LED.
2. I valori angolari finali (`correzionePitch` e `correzioneRoll`) vengono passati alla matrice adattiva del Mixer, che provvede al posizionamento fisico dei servomotori sulle cerniere alari.
3. **Limitazione Termica:** prima della scrittura finale sull'ESC, il comando `comandoGasFinale` viene confrontato con il limite calcolato da `gasMaxTermico()` (§7.4); se lo supera e il flag `sistema_sicurezza_temp` è attivo, viene saturato al valore limite.
4. La variabile `comandoGasFinale` viene trasmessa all'ESC, modulando la trazione in base alle richieste dell'Autothrottle o dello stick manuale.

13.2. Gestione Dinamica dell'Interfaccia Visiva (HMI)

La funzione `gestisci_allarmi()` aggiorna in tempo reale la telemetria visiva *on-board*, utilizzando un'architettura a priorità discendente (*Cascading If-Else*) su tre diodi LED ad alta luminosità:

- **Priorità 0 (Allarme Critico Hardware):** Il sistema verifica i flag di diagnostica di corrente dei quattro servomotori (`estSX_0k`, ecc.). La defezione di un singolo attuatore satura l'interfaccia: i LED Rosso, Verde e Blu vengono accesi simultaneamente a livello HIGH. L'istruzione `return` blocca l'esecuzione della funzione, congelando le luci in questo stato per segnalare l'avaria catastrofica.
- **Priorità 1 (Allarme Logico/Energetico):** Se l'hardware attuativo è intatto, viene valutata la condizione della telemetria radio (`failsafe`) e del voltaggio propulsivo (`batteriaBassa_motore`). Il verificarsi di una di queste criticità accende il LED Rosso, avvisando l'operatore di un degrado operativo imminente.
- **Priorità 2 e 3 (Stati Nominali):** Il LED Verde viene pilotato direttamente dal parsing NMEA del modulo GNSS: si illumina solo se l'oggetto `gps.location.isValid()` restituisce `true`, confermando il *Fix* 3D spaziale. Infine, il LED Blu segnala la transizione modale: si accende se l'operatore ingaggia l'algoritmo di navigazione L1 e il PID (Modalità 2), spegnendosi in caso di volo totalmente manuale.

14. Allocazione dei Comandi e Matrice di *Mixing* Adattiva

Il blocco funzionale preposto all'attuazione fisica della traiettoria è il modulo di *Control Allocation*, comunemente definito *Mixer*. Questa routine riceve in ingresso gli sforzi di controllo calcolati dai loop PID (o i segnali grezzi in caso di volo manuale), espressi come scostamenti angolari desiderati per il beccheggio (δ_{pitch}) e per il rollio (δ_{roll}). Il suo compito primario è tradurre tali richieste in angoli fisici assoluti per i quattro servomotori disposti lungo l'apertura alare.

Ciò che distingue questa architettura da un sistema aeromodellistico tradizionale è la sua natura intrinsecamente adattiva (*Fault-Tolerant Control Allocation*). Il mixer ricalcola dinamicamente la propria matrice di trasformazione in tempo reale, basandosi sui flag di diagnostica hardware e sullo stato energetico del velivolo.

14.1. Risparmio Energetico Attivo (*Load Shedding*)

Ad ogni ciclo di esecuzione, in apertura della funzione `applicaMixer4Servo()`, l'algoritmo valuta lo stato corrente della batteria di servizio dell'avionica (`batteriaBassa_tensy`). Qualora il sistema si trovi in uno stato di *Failover* energetico (ovvero stia prelevando energia dal circuito di emergenza del motore), il firmware forza una procedura di distacco selettivo dei carichi, o *Load Shedding*. Le variabili di stato dei servomotori interni (`intSX_0k` e `intDX_0k`) vengono intenzionalmente invalidate. Questa logica degrada volontariamente la manovrabilità del velivolo per minimizzare l'assorbimento di corrente, preservando i milliampere residui per il sostentamento del microcontrollore, del GPS e del ricevitore radio.

14.2. Matrice Cinematica e Riconfigurazione Strutturale

Il nucleo matematico del modulo analizza le combinazioni booleane della salute degli attuatori (interni ed esterni) ed elabora quattro scenari operativi distinti, modificando l'equazione di governo delle superfici:

- **Caso A (Volo Nominale):** Se tutti i sensori INA219 confermano correnti operative standard, il velivolo sfrutta l'intera aerodinamica. I flap interni vengono dedicati esclusivamente al controllo del beccheggio (funzione di elevatore puro), mentre gli alettoni esterni gestiscono esclusivamente il rollio. Definito $\theta_{centro} = 90^\circ$, la cinematica è:

$$\theta_{int_SX/DX} = \theta_{centro} + \delta_{pitch} \quad (15)$$

$$\theta_{ext_SX/DX} = \theta_{centro} \pm \delta_{roll} \quad (16)$$

- **Caso B (Avaria Superfici Interne):** Se le superfici di coda o i flap interni subiscono un guasto (o vengono disabilitati dal *Load Shedding*), il sistema transiziona

istantaneamente in configurazione *Elevon* (Elevator + Aileron). Le superfici esterne si fanno carico di entrambi gli assi di rotazione:

$$\theta_{ext_SX} = \theta_{centro} + \delta_{pitch} + \delta_{roll} \quad (17)$$

$$\theta_{ext_DX} = \theta_{centro} + \delta_{pitch} - \delta_{roll} \quad (18)$$

- **Caso C (Avaria Superfici Esterne):** Simmetricamente al Caso B, la perdita degli alettoni esterni comporta il trasferimento della matrice di *Elevon mixing* sui servomotori interni, permettendo al drone di mantenere la governabilità completa, seppur con un rateo di rollio ridotto a causa del minore braccio di leva aerodinamico.
- **Caso D (Avaria Totale):** L'assenza simultanea di attuatori validi su entrambi i segmenti alari causa l'interruzione immediata della routine (**return**) per impossibilità meccanica di manovra. Nella versione attuale del firmware questa condizione **non genera** alcun messaggio diagnostico sulla porta seriale: rappresenta un possibile miglioramento futuro da considerare, data la criticità dello scenario.

14.3. Gestione Dinamica dell'Hardware (Attach/Detach)

A valle del calcolo matematico, il firmware implementa un controllo logico sulle istanze degli oggetti **Servo**. Attraverso il monitoraggio dello stato precedente, il sistema invoca i metodi **attach()** e **detach()** solo durante le transizioni di stato.

L'operazione di **detach()** rimuove fisicamente la generazione del segnale PWM sul pin del microcontrollore. Dal punto di vista ingegneristico, questo rende il servomotore guasto "libero" (flottante), azzerando la corrente di mantenimento (*Holding Current*) ed evitando che un attuatore con la cerniera bloccata possa assorbire correnti di corto circuito (fino a 2.5A), le quali causerebbero l'immediato *Brownout* dell'intero sistema.

14.4. Saturazione Meccanica e Attuazione Finale

Prima di procedere all'istruzione di scrittura sui registri hardware (**write()**), i valori angolari elaborati dalle matrici vengono fatti passare attraverso una funzione di vincolo matematico (**constrain**). Tutti gli angoli vengono rigidamente confinati nell'intervallo operativo di sicurezza $[45^\circ, 135^\circ]$. Questa tecnica di *Saturazione Meccanica Software* è indispensabile per prevenire il fincorsa fisico dei leveraggi aerodinamici: tentare di spingere un servomotore oltre i propri limiti strutturali comporterebbe un picco di assorbimento anomalo, che verrebbe immediatamente (ed erroneamente) interpretato dalla routine di diagnostica come un guasto, innescando un falso positivo.

15. Lettura Seriale del Sottosistema GNSS

Ad ogni ciclo del `loop()` principale, il firmware svuota il buffer della porta seriale dedicata al GPS (`GPS_SERIAL`, `Serial1`), passando ogni byte disponibile al parser `TinyGPSPlus::encode()`. Il flusso di dati NMEA viene così decodificato in modo continuativo e non bloccante durante l'intero ciclo di vita del firmware.

Nota Tecnica

La versione attuale del firmware **non implementa** un *watchdog* dedicato sulla porta seriale del GPS (ossia un controllo esplicito, temporizzato dall'avvio, sul numero di caratteri NMEA effettivamente processati per rilevare guasti di cablaggio o baud rate errato). L'unica diagnostica disponibile in fase di volo è indiretta: la validità del *fix* viene verificata tramite `gps.location.isValid()`, usata sia da `aggiornaNavigazione()` sia dal LED verde di stato (`gestisci_allarmi()`). L'aggiunta di un vero watchdog seriale (basato su `gps.charsProcessed()` confrontato con una soglia temporale) rappresenta un possibile miglioramento futuro per diagnosticare guasti di livello fisico (cablaggi TX/RX invertiti, baud rate incompatibile) prima ancora della fase di volo.

15.1. Sistema di Telemetria

All'interno del ciclo principale di esecuzione, il firmware invoca periodicamente la routine `inviaTelemetria()`, responsabile della trasmissione dei dati di stato del velivolo verso la Ground Control Station (GCS) tramite collegamento radio (LoRa).

Per evitare la saturazione della banda trasmissiva e garantire stabilità del link, l'invio dei dati è limitato a una frequenza di 2 Hz (ovvero un pacchetto ogni 500 ms), implementata tramite confronto temporale basato sulla funzione `millis()`.

15.1.1. Acquisizione Dati

Ad ogni ciclo di trasmissione, il sistema effettua una raccolta completa delle grandezze di interesse:

- Tensioni di alimentazione tramite moduli INA219 (motore, logica e servi)
- Assetto del velivolo (pitch, roll, yaw) da IMU
- Altitudine relativa fusa (LIDAR TF-Luna quando disponibile, barometro altrimenti; cfr. § 6.3)
- Velocità (Pitot, GPS e stimata)
- Parametri di navigazione (distanza, rotta ed errore di rotta verso il target)
- Input del radiocomando (canali grezzi)
- Output dei controllori PID

- Stato e posizione dei servomeccanismi, e stato del relè di ridondanza
- Temperature di sistema
- Dati GPS (satelliti, latitudine, longitudine)
- Velocità dal flusso ottico e bitmask di stato dei sensori (flusso ottico, LIDAR, pacchetto S.BUS)
- Limite di gas termico corrente e altitudini grezze pre-fusione (LIDAR, barometro)

15.1.2. Codifica degli Allarmi

Le condizioni critiche del sistema vengono codificate in un intero (`codiceAllarme`) utilizzando una struttura a *bitmask*, che consente la rappresentazione compatta di più stati booleani:

Bit	Significato
0	Failsafe attivo
1	Batteria motore scarica
2	Relè di ridondanza attivato
3	Batteria logica (Teensy) scarica
4	Impatto rilevato
5	Drone in volo

15.1.3. Struttura del Pacchetto

I dati vengono trasmessi sotto forma di stringa seriale delimitata da virgole (*CSV-like*), preceduta da un carattere di start (\$) per facilitarne il parsing lato ricevitore.

La struttura del pacchetto è la seguente:

Indice	Contenuto
0	Start (\$)
1	Modalità di volo
2	Codice allarmi (bitmask)
3–8	Tensioni (motore, Teensy, servi)
9	Stato salute servi (bit concatenati)
10–12	Pitch, Roll, Yaw
13	Altitudine AGL
14–16	Velocità (Pitot, GPS, stimata)
17–19	Navigazione (distanza, rotta, roll target)
20–22	Input radiocomando
23–25	Output controllori
26–29	Posizione servomeccanismi
30–31	Temperature
32–34	GPS (satelliti, latitudine, longitudine)
35	Stato relè ridondanza
36–37	Velocità flusso ottico (X, Y) [m/s]
38	Bitmask stato sensori (flusso ottico / LIDAR / pacchetto S.BUS perso)
39	Errore di rotta [°]
40	Limite di gas termico corrente [µs]
41–42	Altitudini grezze pre-fusione (LIDAR, Barometro) [m]
43	Timestamp locale in ms (<code>millis</code>)
44	Numero progressivo pacchetto TEL1

Pacchetti diagnostici supplementari (TEL2/TEL3/TEL4). Oltre al pacchetto principale TEL1 appena descritto, inviato a 2 Hz, il firmware trasmette in modalità round-robin tre pacchetti diagnostici supplementari, ciascuno contraddistinto da un proprio carattere di start e inviato uno alla volta ogni 2 secondi circa (o immediatamente su richiesta del comando REQ_DIAG, § 2.2):

- **TEL2 (\$2,)** — correnti e potenze delle batterie motore e Teensy, correnti dei quattro servomotori, flag di batteria scarica, conteggi grezzi del flusso ottico (`dx`, `dy`), comando gas pre-limitazione termica e stato della limitazione termica.

- **TEL3** (\$3,) — diagnostica completa dei quattro controllori PID: errore, termine proporzionale, integrale e derivativo per ciascun asse (Altitudine, Beccheggio, Rollio, Autothrottle), pitch target risultante dal loop di quota, quota e velocità target correnti.
- **TEL4** (\$4,) — dati GPS estesi (altitudine, rotta, flag di validità), pressione barometrica assoluta, tara altimetrica, lettura grezza e zero calibrato del Pitot, offset di tara IMU, accelerazione lineare e velocità angolare sui tre assi, livelli di calibrazione BNO055, e i canali radiocomando dal 4° al 16° (non presenti in TEL1).

Un contatore interno seleziona ciclicamente TEL2, TEL3 e TEL4 a ogni invio diagnostico successivo, mantenendo così ad alta frequenza (2 Hz) i soli dati critici di volo (TEL1) e relegando i dati diagnostici a una cadenza più contenuta per non saturare la banda LoRa.

15.1.4. Esempio di Pacchetto Telemetrico

Si riporta di seguito un esempio completo e coerente di pacchetto telemetrico, costruito secondo la struttura a 45 campi (indici 0–44) definita in § 15.1.3 e la funzione `inviaTelemetria()`, con i valori di gas espressi in microsecondi come da § 11.2.1 e § 12.2.2:

```
% $,2,4,14.80,5.02,5.10,5.08,5.12,5.09,1111,2.3,-1.5,178.2,12.5,45.2,
43.8,44.5,120,185.4,10.2,1500,1498,1200,10,-5,1300,90,92,88,91,45.6,
22.3,12,41.902782,12.496366,1,-1.00,-1.00,3,7.2,2000,5.80,12.50,1453200,
42
```

Valore	Indici	Descrizione
\$	[0]	Carattere di start, indica l'inizio del pacchetto
2	[1]	Modalità di volo (2 = Auto GPS)
4	[2]	Codice allarme bitmask (bit 2 attivo → relè di ridondanza inserito)
14.80	[3]	Tensione batteria motore, V
5.02	[4]	Tensione alimentazione Teensy, V
5.10 ... 5.09	[5–8]	Tensioni linee servi Int SX / Int DX / Est SX / Est DX, V
1111	[9]	Stato salute servi concatenato Int SX / Int DX / Est SX / Est DX, tutti funzionanti
2.3	[10]	Pitch reale, gradi
-1.5	[11]	Roll reale, gradi
178.2	[12]	Yaw/heading, gradi
12.5	[13]	Altitudine relativa <code>G_altitudine</code> , m — quota fusa LIDAR/Baro
45.2	[14]	Velocità aria Pitot, km/h
43.8	[15]	Velocità suolo GPS, km/h
44.5	[16]	Velocità stimata fusa, km/h
120	[17]	Distanza dal target, m
185.4	[18]	Rotta verso il target, gradi
10.2	[19]	Roll target calcolato dalla navigazione L1
1500, 1498, 1200	[20–22]	Input radiocomando grezzi — pitch, roll, gas, unità S.BUS [172, 1811]
10, -5, 1300	[23–25]	Output controllori — PID pitch (gradi), PID roll (gradi), gas finale in μs
90, 92, 88, 91	[26–29]	Posizione fisica attuale servi Int SX / Int DX / Est SX / Est DX, gradi
45.6	[30]	Temperatura motore, °C
22.3	[31]	Temperatura avionica/barometro, °C
12	[32]	Numero satelliti GPS agganciati
41.902782	[33]	Latitudine
12.496366	[34]	Longitudine
1	[35]	Stato relè di ridondanza, attivo — alimentazione su BEC
-1.00	[36]	Velocità X flusso ottico, m/s (sentinella: quota oltre AL- TEZZA_MAX_SENSORE_OTTICO = 4.0 m, misura non

Osservazioni

Il pacchetto presenta una struttura deterministica a 45 campi (indici 0–44) con ordine fisso, che consente un parsing robusto lato ricevitore. L'utilizzo di valori numerici in formato ASCII separati da virgole rappresenta un compromesso tra leggibilità, semplicità implementativa ed efficienza trasmissiva. A differenza delle revisioni precedenti del firmware, il campo di output gas (indice 25) e i valori di gas riportati in telemetria sono ora espressi in microsecondi di impulso PWM, in coerenza con l'intera catena di comando motore basata su `writeMicroseconds()` (cfr. § 11.2.1, § 12.2.2).

L'esempio mostrato evidenzia una condizione operativa nominale con sistema in volo automatico, tutti i servi funzionanti, quota fusa LIDAR/barometro e relè di ridondanza attivo, indicativo di una precedente condizione di sottotensione sulla linea primaria.

15.1.5. Formato e Robustezza

Ogni valore numerico viene trasmesso con una precisione controllata (tipicamente 1–2 cifre decimali) per ridurre il payload e migliorare l'efficienza trasmissiva.

In assenza di segnale GPS valido, il sistema trasmette valori nulli predefiniti (0, 0.000000, 0.000000) per mantenere invariata la struttura del pacchetto e garantire la robustezza del parsing lato GCS.

15.1.6. Considerazioni Sistemistiche

L'approccio adottato presenta i seguenti vantaggi:

- Basso overhead computazionale e semplicità di parsing
- Elevata compatibilità con sistemi di monitoraggio e logging
- Robustezza agli errori grazie alla struttura fissa del pacchetto
- Scalabilità per l'aggiunta di nuovi parametri

La telemetria rappresenta quindi un elemento fondamentale per il monitoraggio in tempo reale, la diagnostica e la sicurezza operativa del sistema UAV.

Modulo / Sensore / Costante	Collegamento / Significato
TinyGPSPlus gps	Collegato a Serial1 (GPS_SERIAL), baud 9600
Adafruit_BNO055 giroscopio	I2C default, indirizzo 0x28
Adafruit_BMP3XX barometro	I2C default
PIN_T_motore	A12
Global_Temperatura_motore	variabile per la lettura temperatura motore
temp_aria_barometro	variabile per temperatura barometro
TELEMETRIA	Serial4
SBUS ricevente	Serial7
canaliRC[16]	array per canali ricevuti dalla SBUS
PIN_ARIA	A0 (Pitot)
DENSITA_ARIA (valore iniziale)	1.225 kg/m ³ — ricalcolata a ogni ciclo da aggiorna_densita
R_SPECIFIC	287.05 J/(kg · K)
FATTORE_CONVERSIONE_PA	3.22
pinIntSX	6 (servo flap interno SX)
pinIntDX	22 (servo flap interno DX)
pinEstSX	23 (servo flap esterno SX)
pinEstDX	24 (servo flap esterno DX)
pinMotore	10 (servo motore)
servoInternoSX	Flap interno sinistro (pitch)
servoInternoDX	Flap interno destro (pitch)
servoEsternoSX	Flap esterno sinistro (pitch + roll)
servoEsternoDX	Flap esterno destro (pitch + roll)
motore	Servo motore principale
sensoreMotore	INA219 0x40
sensoreIntSX	INA219 0x41
sensoreIntDX	INA219 0x42
sensoreEstSX	INA219 0x43
sensoreEstDX	INA219 0x44
sensoreTeensy	INA219 0x45
VALORE_BATT_MOTORE_BASSA	13.5 V
VALORE_BATT_TEENSY_BASSA	4.9 V
SOGLIA_G_SCHIANTO	50 m/s ²
CENTRO_SERVO	90°
MAX_ROLL	35°
MAX_PITCH	20°
ALTEZZA_MAX	120 m
ALTEZZA_MIN	10 m
GAS_NEUTRO	1000 μs
GAS_MINIMO	1200 μs
GAS_CROCIERA	1450 μs